Grant FFI 2013-01296

Synligare utveckling av inbyggda fordonssystem – visualiserad kravhantering och samverkan

Visibler Development of Embedded Automotive Systems - Visualised Requirements Management and Collaboration

Report type — Deliverable D1.1

Report name — Needs Identification

Dissemination level	**Public**
Status	**Final Release**
Version number	**2.0**
Date of preparation	**2016-05-20**

Authors

Editor	**E-mail**
Henrik Lönn	henrik.lonn@volvo.com

Authors	**E-mail**
Urban Ingelsson	urban.ingelsson@semcon.com
Henrik Lönn	henrik.lonn@volvo.com
Magnus Skoog	magnus.skoog@autoliv.com

The Consortium

Volvo Technology Corporation AB - Autoliv AB - Arccore AB- Systemite AB - Semcon Sweden AB

Table of contents

Terminology

Below is a set of terms used in project Synligare. The list largely excludes metamodel elements from AUTOSAR and EAST-ADL, as these are defined in the respective specification.

Term	Description
AUTOSAR	Software platform for automotive software, and description format allowing representation of application software and configuration of the platform
Artifact	A general term for engineering elements that compose a solution
Component	A general term for software, hardware and functional components
DIA	Development Interface Agreement
EAST-ADL	Architecture Description Language for capturing system level aspects of automotive embedded systmes
Element	A general term for parts of a model.
Embedded system	Software and electronics embedded in a product
File format	A file format makes information storable and machine readable. Storing information in a file that complies with a file format makes it possible for a sw tool to read it. It does not guarantee that the information is structured. Example: c++ code can be stored in a textfile or in a MS Word document.
FROP	Feature Roll Out Plan
Function Point	A measurement of business functionality
Information model	A user level model representing subject elements and their relations
Legacy tools	Tools that already exist in a specific context
Meta-model	A pattern for representing user level models
Language	A language defines syntax and semantic sufficient to capture its intended scope
Metric	A quantification of aspect(s) of a design, system, project, organization , etc.
OEM	Original Equipment Manufacturer
Ontology	An ontology is more than a language, because it contains defined concepts. An ontology is stricter than a natural language, because it aims to be unambiguous.
Requirement	A singular documented physical and functional need that a particular design, product or process must be able to perform
Requirement management	The process of documenting, analyzing, tracing, prioritizing and agreeing on requirements and then controlling change and communicating to relevant stakeholders.
ReqIF	An exchange format for requirements
RFI	Request for Information
RFQ	Request for Quotation
Schema	A schema defines the structure of a database or data file.
SEooC	Safety Element out of Context
SIL	Safety Integrity Level

Synligare	FFI Project Grant FFI 2013-01296
System Model	A set of elements and relations that represents a system
TRL	Technology Readiness Level, a measure introduced by NASA to characterize technology maturity from 1 (initial concept) to 9 (deployed technology)
Tier-n Supplier	A supplier that provides components directly to an OEM (Tier-1) or Tier-i supplier (tier-i+1)
UML	Unified modelling language, a generic modelling notation
User Stories	Short description of functionality in informal language
View	A representation of a whole system (or aspects thereof) from the perspective of a related set of concerns
Viewpoint	A specification of the conventions for constructing and using a view.
XML	A file format based on tagged data elements in text files
Work product	The tangible result of a particular methodology or process step
Well-formed	The property of a model that it complies with applicable syntactical rules

1 Introduction

This chapter introduces the document by defining purpose and scope and providing a background to the addressed topics.

1.1 Purpose and Scope

The purpose of this document is to document the conclusions from the first work package (WP1) of project Synligare, WP1. The report identifies needs and requirements relevant for the project.

The goal of WP1 is to:

1. Define from the perspectives of OEMs and Tier-1s what is complex and hidden today
2. Define useful metrics and views from a user and collaboration perspective
3. Analyze this from an information perspective and identify which type of information is needed

In particular WP1 addresses the project goals to identify

- 5 metrics for characterization and follow-up of development of software-based systems
- 5 relevant views providing overview on a complex requirement set and activities related to requirements management

The remainder of this chapter describes:

- in Section 1.2 the observed problems in which Project Synligare finds its context,
- in Section 1.3 the intentions of Project Synligare in terms of
  - the method for defining views and metrics supported by structured information models,
  - a twofold vision, to automate generation of view and metrics, and to enable early feedback through validation and verification of early work products
  - the collaboration that the structured information models, views and metrics will enable
  - how to increase productivity by enabling synthesis of work products and information that helps to identify and avoid problems, as well as to plan appropriately
  - how to address safety and quality concerns
  - the tools that will be demonstration platforms for the results of Synligare

1.2 Common Issues in System Development

Embedded system development is a highly complex process, which many challenges. Fast, efficient and correct product development is challenged by increasing complexity. Below we will list some of the concrete challenges deemed to be within the project scope.

- A large number of requirements are used to characterize systems.
  The sheer volume hampers understanding and overview of system specifications.
- Architecture descriptions are being formulated as textual requirements.
  Textual requirements are used where structural models would be more appropriate.
- Mature systems have many system requirements to capture legacy properties. This adds to the number of requirements, and makes it difficult to distinguish between fundamental requirements, and solution-dependent constraints.

- Many elements of a system are typically reused.
  Reused components carry a lot of design decisions that may be in conflict with the new environment.
- The delay from initiating a change until development work is completed is long.
  Integration and verification often happens long after the initial design decisions are taken, making changes difficult and expensive.
- The complexity and heterogeneity of systems makes them difficult to cover with a consistent specification and model.
- Planning, specifications and processes change frequently.
  Changes increase the difficulty in maintaining correct and consistent system specifications.
- Short-term results are prioritized over long-term benefits.
  Prioritizing short term time and cost may cancel activities that pay off later in the life-cycle or in the following project.
- Development organizations are complex.
  Because of the supply chain in the automotive industry, double work occurs when OEM and Tier-1 specify and re-specify the same system.

1.3 Project Intentions and Constraints

To address these issues and difficulties, Synligare will investigate solutions that allow OEMs and Tier-1s to collaborate efficiently. Means for requirements to be visualized, analyzed and organized together with the system structure will be proposed to make engineering information more tractable. A demonstrator system is used to set the proposed enhancements on test in a realistic situation. The intended method for the project is to identify views and metrics that bridge complexity and enable efficient collaborative product development, such that the information required for those views and metrics can be identified and modeled in structured information models. These structured information models are to be used to standardize how specifications are written and enable effective tool support for automation of views, metrics and work products. The intended outcome is that OEMs and Tier-1s will collaborate more efficiently even on complex products.

To limit scope, focus will be on the specification, design and planning, assuming change management is handled by other means.

1.3.1 Vision

The first part of the vision for Synligare is to make project development more concrete via more efficient views and metrics. This generally requires additional information in a system model, see Figure 1.

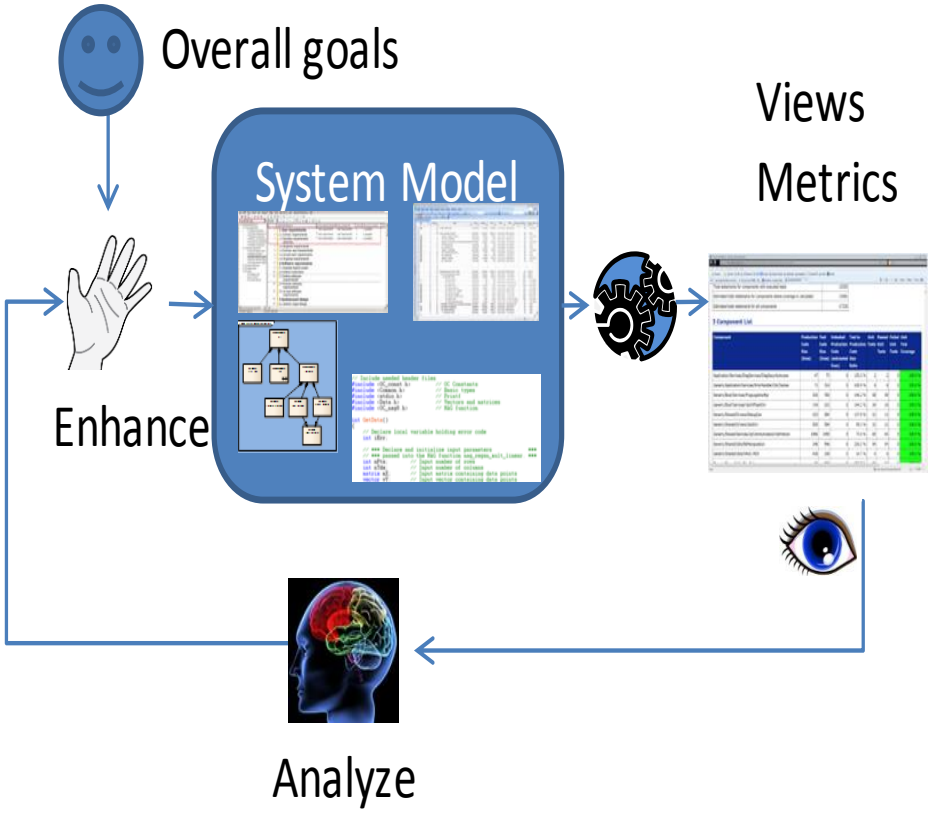

Figure 1. Improving system definition based on views and metrics

Formalization of system specification allows efficient views and metrics to be calculated. On this basis, engineers can analyze and improve the solution.

Another part of the vision is the use of early-phase integration, validation and verification, see Figure 2. By using abstract models, system specifications can be defined without implementation details to quickly identify the fundamental design and any fundamental issues in the intended solution. The engineering information can be captured with appropriate separation of concerns, avoiding implementation details from obscuring high-level design and requirements. A proper organization of information makes contradictory architectural requirements identifiable.

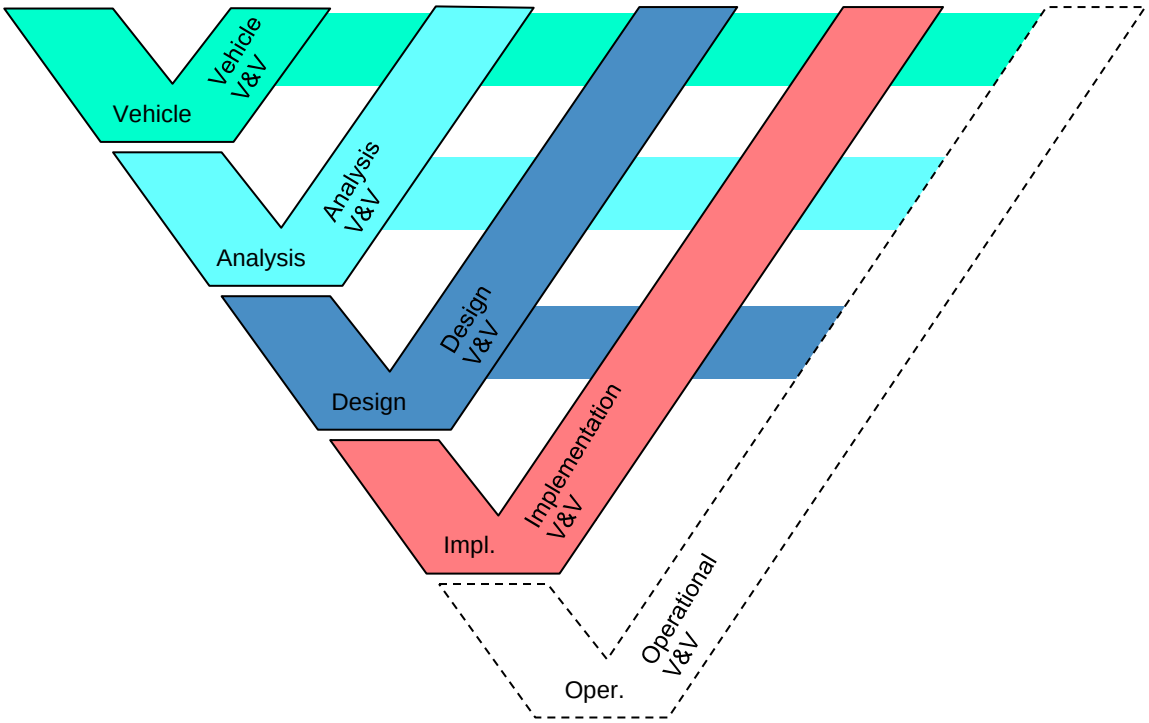

Figure 2. Use of early-phase V&V to frontload the engineering effort

1.3.2 Collaboration

OEM-Tier-1 collaboration is a focus area of Synligare. Using an agreed exchange format and applying effective views and metrics, collaboration will be more efficient, faster and with better quality and safety. Figure 3 shows how such collaboration scenario may look.

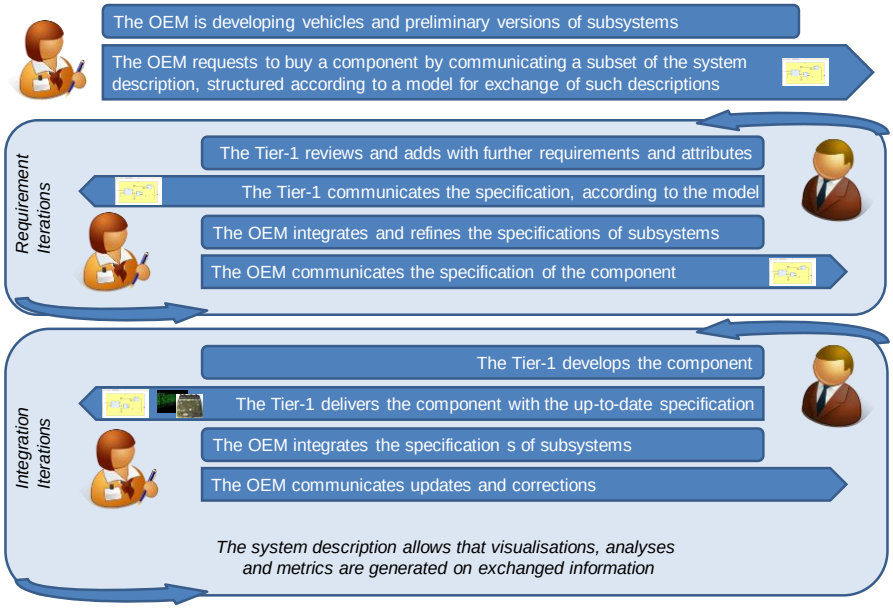

Figure 3. Collaboration Scenario

1.3.3 Productivity

By introducing automation of tedious specification tasks, such as automatic synthesis of information for detailed models based on overall specification, the productivity increase and more time can be spent on iterative improvements.

Examples of automation includes

- Fast feedback
  Productivity is improved if design decisions result in immediate feedback.
- Requirements
  Adding a requirement may require linking to other requirements and to verification procedures. This can be automated if the related elements are identified by the engineer.
- Component synthesis (top-down, bottom up) and reorganization
  Auto generation of initial models increase productivity and avoids simple mistakes. An example of reorganization support is automatically moving ports and connectors when connected elements are moved.
- Verification and Validation (V&V) structure synthesis.
  The information structure capturing V&V information can be auto generated, and even the actual test stimuli.
- Consistency checks
  Manual review can be assisted by automatic check of well-formedness and other syntactic and semantic checks. Consistency criteria can be applied within a solution or between solution and requirements

1.3.4 Safety, Quality and Performance

Defining the right product requires proper management and improvement of safety, quality and performance. This includes relevant analyses and artifacts that can be feasibly synthesized to automate otherwise manual software development tasks.

To provide benefit in terms of making systems development predictable and compliant with safety efforts, quality efforts and performance efforts, suggested analyses and synthesis automations will be chosen such that the requirements can be communicated unambiguously and understandably.

- Identification of relevant visualizations
- Identification of relevant formalizations
  - E.g. for simulations, virtual prototyping, rapid prototyping, ...
- Identification of relevant means for communicating during several phases:
  - Request for Information (RFI)
  - Request for Quotation (RFQ)
  - Contract
  - Iteration of specification between OEM and Tier-1
  - Safety Element out of Context (SEooC)
  - Development Interface Agreement (DIA)

1.3.5 Tool Platform

Synligare will build upon existing tools and contribute to extensions of selected tools as follows

- Implementation of one or more extensions to SystemWeaver
- Implementation of one or more extensions to EATOP
- Implementation of export functionality on selected Legacy Tools, and possibly other extensions to Legacy Tools.

1.4 Report Outline

This report is further structured as follows. Chapter 2 gives the background for Project Synligare, in terms of state-of-practice, the latest developments in system description modelling and related on-going projects. Chapter 3 describes the methods that have been applied to identify views and metrics, including questionnaires conducted with requirements engineers and meetings conducted with a tool vendor and stakeholders responsible for quality. Chapter 4 describes the metrics that have been suggested and categorizes them with respect to selection criteria for useful views and scenarios in which they can be useful. Chapter 5 describes viewpoints that have been suggested, as well as filters and layout strategies that turn the viewpoints into useful views. The viewpoints are categorized with respect to selection criteria and scenarios in which they can be useful. Chapter 6 describes how structured information models are populated into specifications and how these specifications are communicated. Chapter 7 describes how collaboration on specifications can be conducted with regard to liability, changes and information propagation in a supply chain or a development team as the developed product moves through its lifecycle. Chapter 8 discusses requirements on tool prototypes that are to be developed in Synligare. Chapter 9 describes scenarios in which the results of Synligare can be demonstrated. Chapter 9.16 concludes this report by listing the selected views and metrics.

2 Background

This chapter provides the background for the work performed in WP1, in terms of existing efforts at project parts and others to improve collaborative product development and address requirements engineering issues.

2.1 State-of-Practice

There are already advanced methods and tools in full scale use at Volvo GTT and other OEMs as well as with Autoliv and other Tier-1s. The Synligare project seeks to meet this state-of-practice in the context of an open representation format, and with a generic tool set. For this reason, the current approach at Volvo GTT and Autoliv is characterized below.

Volvo GTT captures the complete EE architecture in models, describing functional components with inputs and outputs, and their allocation to the hardware topology. Requirements are allocated to these components, but also internally related to trace requirement derivation from top level requirements. Various domain-specific tools are used to detail and analyze the design. Requirements and specifications are largely document-based in their final and archived form.

Autoliv system development is based on current best practice. A process is being established following SPICE level 3 and ISO26262 ASIL-B. This process is document driven, but includes models in tools that are domain specific. As requirement tool DOORS is used and Enterprise Architect is used as a design tool. A safety tool is also used to analyze fault propagation and dependability. The code is written from design and requirements and according to coding guideline, but no code is generated. The code is verified according to requirements and design. The planning is done in traditional tools like Microsoft Project.

2.2 Requirements Engineering Challenges from Functional Safety

To minimize the risk that faults in automotive embedded systems cause accidents, whether by wear-out, bad design, malpractice or other reason, functional safety has become important. Standardization such as ISO 26262 Functional Safety – Road Vehicles makes it possible to find a state-of-the-art level of engineering practice to measure and talk about functional safety. It is all-encompassing, influencing almost all activities in development of automotive embedded systems. In particular it involves analyzing each function with regard to how it can lead to accidents and find ways to minimize the risk that those accidents occur. It can be seen as an unbroken chain of requirements, analyses, reviews and design decisions from safety goals to lines of software code or lines on a schematic. There are recommended methods and analyses that should be applied during the development. These should be traceable to requirements structured in a hierarchy. On each level of the hierarchy, the requirements should attain a list of quality attributes. Each safety-related requirement should be verified.

Functional safety influences Project Synligare in three aspects:

- Views and metrics that support the activities to achieve functional safety are of high importance
- The structured information models for representing requirements should support the traceability to and from levels of hierarchy, architectural components, analyses, reviews, design decisions, verification cases, etc.
- The structured information models should be appropriate for automating activities to achieve functional safety.

Concrete examples of views that concern functional safety

- Traceability

- Overview of activity progress, e.g. review, verification, etc.
- Analysis views, e.g. FMEA, FTA, etc.

Concrete examples of metrics that concern functional safety

- Verification coverage
- Reliability
- Remaining risk
- Common cause failure probability

2.3 EAST-ADL Approach

Synligare will rely on a model-based approach where the relevant information is captured in an integrated model. For this reason, the EAST-ADL is characterized below.

EAST-ADL represents an Architecture Description Language (ADL) initially defined in the European ITEA EAST-EEA project. It was subsequently refined and aligned with the modelling approach of the AUTOSAR automotive standard [2] in national and international funded projects including the ATESST and MAENAD projects [1], [5]. It is maintained by the EAST-ADL Association [3].

EAST-ADL is an approach for describing automotive systems through an information model that captures engineering information in a standardized form. Aspects covered include vehicle features, functions, requirements, variability, software components, hardware components and communication.

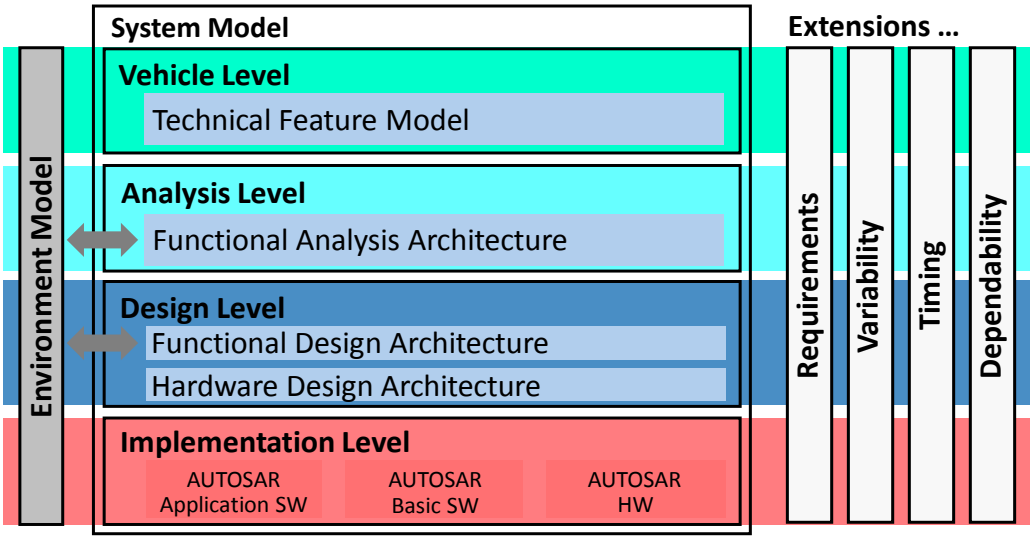

Figure 4: The EAST-ADL's breakdown in abstraction levels (vertically) and in core system model, environment and extensions (horizontally).

The functionality of the vehicle is described at different levels of abstraction and in different parts.

The four abstraction levels covered by the EAST-ADL are (see Figure 1):

- **Vehicle Level**
  Feature trees characterizing the vehicle content as it is perceived externally.
- **Analysis Level**
  An abstract functional architecture defining systems from a functional point of view.

- **Design Level**
  The detailed functional architecture allocated to hardware architecture.
- **Implementation Level**
  The implementation of the embedded system represented using AUTOSAR elements

Each of the abstraction levels has a specific role. From the Vehicle Level stating *what* the vehicle should do through Analysis, Design and Implementation levels that define, at various level of abstraction, *how* this is done. Features on the Vehicle Level allow the organization of vehicle content in a solution-independent way. Requirements can be linked to features such as markets and brands as well as to technical features such as wipers or brakes.

The proposed abstraction levels and the contained elements provide a separation of concerns and an implicit style for using the modelling elements. The system description is complete on each abstraction level with respect to the concerns of that level.

The EAST-ADL extensions include requirements, variability, safety, behavior, timing, and generic constraints. Such elements reference the core elements at all abstraction levels. The behavior extension support modes, which allows different requirements, behaviors and constraints to be active at different time instants.

The core model at the Vehicle Level is organized around *Features* in the *Technical Feature Model.* These represent the vehicle from a top-level perspective without exposing the realization. It is possible to manage the content of each vehicle and entire product lines in a systematic manner using sets of Feature Models with internal relations like needs and exclude, and configuration decisions across Feature Models.

A complete representation of the electronic functionality in an abstract form is modelled in the *Functional Analysis Architecture* (FAA). One or more entities (Analysis Functions) of the FAA can be combined to realize Features in the *Technical Feature Model*. The FAA captures the principal interfaces and behavior of the vehicle's subsystems. It allows validation and verification of the integrated system or its subsystems on a high level of abstraction. Critical issues for understanding or analysis can thus be considered, with less risk of obscuring by implementation details.

The implementation-oriented aspects are introduced while defining the *Functional Design-Architecture* (FDA). The Features are realized here in a function architecture that takes into account efficiency, safety, legacy and reuse, purchasing strategy, hardware allocation, etc. The function structure is such that one or more functions can be subsequently realized by one or more AUTOSAR software components (SW-C). The external interfaces of such components correspond to the interfaces of the realized functions.

The *Hardware Design Architecture* (HDA) represents the topology of the physical elements. It forms a constraint for development and the hardware and software development needs to be iterated and performed together. There is also an indirect effect of hardware on the higher abstraction levels. Control strategies or the entire functionality may have to be revised to be feasibly allocated to the actual hardware architecture. This reflection of implementation constraints needs to be managed in an iterative fashion.

The representation of the system implementation, the *software architecture* and its allocation details to the embedded system, is not defined by EAST-ADL but by AUTOSAR. Traceability is supported from implementation level elements (AUTOSAR) to Vehicle Level elements. Further, the EAST-ADL extensions for ISO26262, requirements, variability, etc. can be applied to the AUTOSAR elements. Traceability through the extension elements is applicable through the abstraction levels all the way down to implementation level.

As a complement to the above, an *environment model* is required for verifying and validating Features across all abstraction levels. Verification could for example be carried out using simulation or formal analysis techniques.

The Environment Model, sometimes referred to as plant model, captures the behavior of the vehicle dynamics, driver, etc. It represents all relevant elements interacting with the vehicle systems defined in the previously described architectures. Examples include a) the mechanical and hydraulic systems in the vehicle, b) the near environment including road surface and adjacent vehicles, and c) the far environment such as road traffic informatics.

Different tasks will typically require different detail and scope of the Environment Model. For example, there may be a detailed vehicle and powertrain dynamics model to assess gear change performance and a low-detail dynamic model for fuel consumption assessment. The alternatives are selected using the variant management extension or using alternative Environment Models for each task. The same Environment Model can typically be used across several abstraction levels.

2.4 Project Verispec

The FFI project Verispec addresses use of formal methods to specify key aspects of automotive systems. By proposing appropriate means to work with formal specifications and related verification methods, correctness and safety can be secured. This is an important contribution to meeting the requirements of the ISO26262 functional safety standard.

Because Synligare defines an overall approach to representing engineering information in models, the VeriSpec technology can be applied in the same framework. Together, the projects can find means to avoid errors in specifications while identifying an efficient way of working across companies.

3 Methods to Find Efficient Views and Metrics

The method is based on two parts. The first is to study earlier academic work. The second is to get best practice from ongoing and completed projects at the OEM and suppliers.

3.1 Literature Overview

A literature overview is given in Appendix C . This overview contains references to academic literature that mention using views and metrics in requirements engineering, as well as references to commercial software tools that employ views and metrics. Further there are references to academic literature that mention use of structured information models.

3.2 Method for Obtaining Best Practice

Two questionnaires were created to gather information on the current state of requirements management in the organizations represented in project Synligare. Further, a meeting with an external tool vendor LEQM was conducted to learn about their meta-modelling work and how they work with views and metrics. Another meeting was conducted with engineers at Volvo GTT that are involved in purchasing subsystems from suppliers to learn about the issues involved in collaborative systems development. This section will report on the questionnaires, the experiences of a tool vendor, and the experiences of collaborative systems development.

3.2.1 Questionnaire 1

The first questionnaire was intended to find out from engineers at Semcon what according to their experience are relevant problems in requirements engineering and in particular their experiences from (lack of) views and metrics. The questionnaire was designed with some consideration of easily accessible guidelines for how to formulate questions such that they can be understood and be acceptable to the engineer that is responding. The questionnaire had in the order of 50 questions distributed over the following parts:

- General problem identification
- Views
- Metrics
- Visualization
- Use of structured information models

Four engineers responded to this first questionnaire. The outcome of processing the responses is listed in Appendix A . A summary of that outcome is listed here.

The present state of requirements engineering is as a time consuming task that is often neglected and that key stakeholders have low awareness of. Key problems arise from the quantity and lack of overview over requirements. Reuse of requirements is typically difficult. Key solution approaches are to improve communication between stakeholders affected by the requirements to improve understanding, terminology, coordination and change management. Activities that the engineers that responded find significant to support are early stages of requirements engineering, verification and validation. The responses indicate that engineers like to configure their own views, metrics and tool automations as they endeavor to do their assigned tasks better and easier.

The metrics and views that were suggested in the responses were further specified and categorized in Chapter 4 and Chapter 5, respectively. For project Synligare, the responses from the questionnaire are seen as suggestions.

3.2.2 Questionnaire 2

The second questionnaire was based on the first questionnaire but was adapted as a material to support in-depth interviews with engineers at Autoliv.

The questionnaire had in the order of 30 questions distributed over the following parts:

- Handling of customer, system and software requirements
- Handling of system and software architecture and design

Five engineers responded to this questionnaire. The outcome of processing the responses is listed in Appendix B . A summary of that outcome is listed here.

The RFQ process today is time consuming. OEM requirements are on a much more detailed level than expected from a stakeholder requirement. This leads to additional (unnecessary) requirement work on other levels.

There are paradigms that make it hard to work efficient. The paradigms have their own views, data set / models:

- Purchase: Focus on RFQ/Customer requirements
- Project Management: Traditional plans and User Stories/FROPs
- System Engineer: Focus on system requirements
- Architects: Work with architecture models
- SW engineer: Focus on code and are reluctant to write software requirements

The planning and prioritization is not based on requirements. Instead other means are used, like FROPs and User Stories.

The work with requirements and design is suffering from the short time frame where release plans have the highest priority.

It is hard, close to impossible to maintain requirements, design and code simultaneously.

3.2.3 Best Practice of Using Views and Metrics in a Planning Tool

As an example of how views and metrics are used today to increase the value of a tool used in the automotive industry, we studied Virtual Project Leader (VPL) from LEQM. This tool is today used at Volvo Car Company to plan development projects.

The tool presents a hierarchy of system components and their requirements. This data is actually stored in a database. In a corresponding hierarchy of views, the development plan is shown, on a top level for the complete car, on a second level for systems and further down in the hierarchy, units to be bought from suppliers, so called component assignments (komponentuppdrag). Each entry in the hierarchy that represents the plan has a pie diagram with various shades of green for components that are according to plan or completed, and shades of red for components that have delays or other problems. Each entry also has a chart that shows the parts of the component in question and their statuses over time. In VPL, the plan is represented on such a time scale that the smallest time unit shown is a week. In each view, for the various steps of the hierarchy, it is possible to simply click in the view to reach the next level of hierarchy.

From the database, project requirements are exported, consisting of a combination of the system requirements and the development plan. Such project requirements for certain units are sent to

suppliers or project teams, and there is also ways of importing their status reports and integration data into VPL.

In VPL, much of the data processing is automated based on templates and the basic elements of the views are static, such that they can be generated automatically. From those basic elements, like existing pie charts, a custom view can be created and reused in other projects. This custom view is called “status dashboard”. Deviation reports are given as lists of text. There is a one-click report and possibility to view the plan as a Gantt chart.

Further, it is possible to configure what statuses a component can have and what colors should be associated with that status. For example, a reference project defined 10 possible states for an activity, namely

- Not planned
- Not ok
- Not confirmed
- After deadline
- Proposed date
- Ongoing
- Due later
- Deviation agreed
- N/A
- OK

Such details are kept in reference documents for milestones and activities. A typical development project for an embedded system has about 300 activities. A development project for a purely mechanical system can have about 40 activities.

The VPL tool can interface or change data with other tools, such as verification tools, to trace statuses and progress reports. However, this interfacing and data exchange is for now mostly to trace the time plan. Error reports are not included, since LEQM has for now made the decision not to connect too many dependencies to a plan. Therefore, it is for now not possible to see a view that shows what rescheduling that might need to be done.

All the requirements that are in the tool have been elicited in some other tool.

Both OEM and Tier-1 can use VPL, but they would see different sets of views.

In summary, VPL is a good example of how views and metrics are used to support leading a development project according to its time plan in collaboration between OEM and Tier-1. At this time, the templates and reference documents are the closest thing to a meta-model or structured information model. These templates and reference documents are typically not shared with other tools, which is in contrast with the purpose of project Synligare to enable collaboration by linking various tools using a common structured information model. The visual aspects, of the hierarchy of views and the graphs, presented by VPL are inspiring examples of how overview can be achieved and how a tool can guide the user to relevant information. The possibility to define a custom dashboard is positive, but that possibility is limited by the user interface and not by the underlying information model.

3.2.4 Best Practice of Collaborative Systems Development

To get the OEMs perspective on collaboration between OEM and Tier-1, two engineers from Volvo GTT were asked to present to project Synligare about their way of working. A typical development project has four milestones and three phases, as follows

- Milestone 1 – Main Requirement Baseline

    - Pre-study and system development phase
        - Pre-study
        - Concept development (Technical report)
        - System development
- Milestone 2 – System Specification Baseline
    - Implementation phase
        - Component development
        - System integration
- Milestone 3 – System Integration Baseline
    - Integration and verification phase
        - System testing
- Milestone 4 – System Release Baselining

Typically, each phase runs in even multiples of 16 weeks, making it possible to pipeline activities over expert teams.

Systems are developed in programs and concepts for systems are developed in projects.

If the development process is seen as the classical V-diagram, collaboration between OEM and Tier-1 starts in the specification phase, at a point called system specification baseline (SSBL), when requirements are released (called RecTrace1) and delivered to a supplier. In the ongoing implementation at the suppliers, specification can in certain aspects continue and as issues arise, change requests are submitted and processed. At the end of the implementation phase, a requirement specification baseline is recorded (called ReqTrace2). After some verification of the system at the suppliers, a further baseline of the requirement specification (called ReqTrace3) is delivered from the supplier to Volvo GTT. However, it is not until the validation phase at Volvo GTT, at the point called system integration baseline (SIBL), that ReqTrace3 is compared against the original requirements.

Reports are possible on system specification, component and parts. As the system is being developed, it reaches various levels of maturity that enables different reports.

Issues with today's way of working include the following

- Change requests are rarely back-annotated into the system specification.
- Some reports issued at the suppliers are inaccessible for Volvo GTT
- Sometimes the wrong template is used for creating the reports
- It is not until the validation phase that the requirement specification from the supplier is compared with the original specification.

It is difficult to optimize the collaboration between OEM and Tier-1 with this setup, since there is no real possibility to do fork and merge of requirement specifications and there traceability is not enough to enable proper impact analysis.

3.3 Lean Principles and Views

Inspired by Lean development [7] the goals for project Synligare could be formulated like:

- Minimize waste

- Simplify collaboration

To work in a lean manner means that you minimize waste. Figure 5 illustrates this. By focusing more on detailed planning and design, some waste like late re-planning and bugs are reduced.

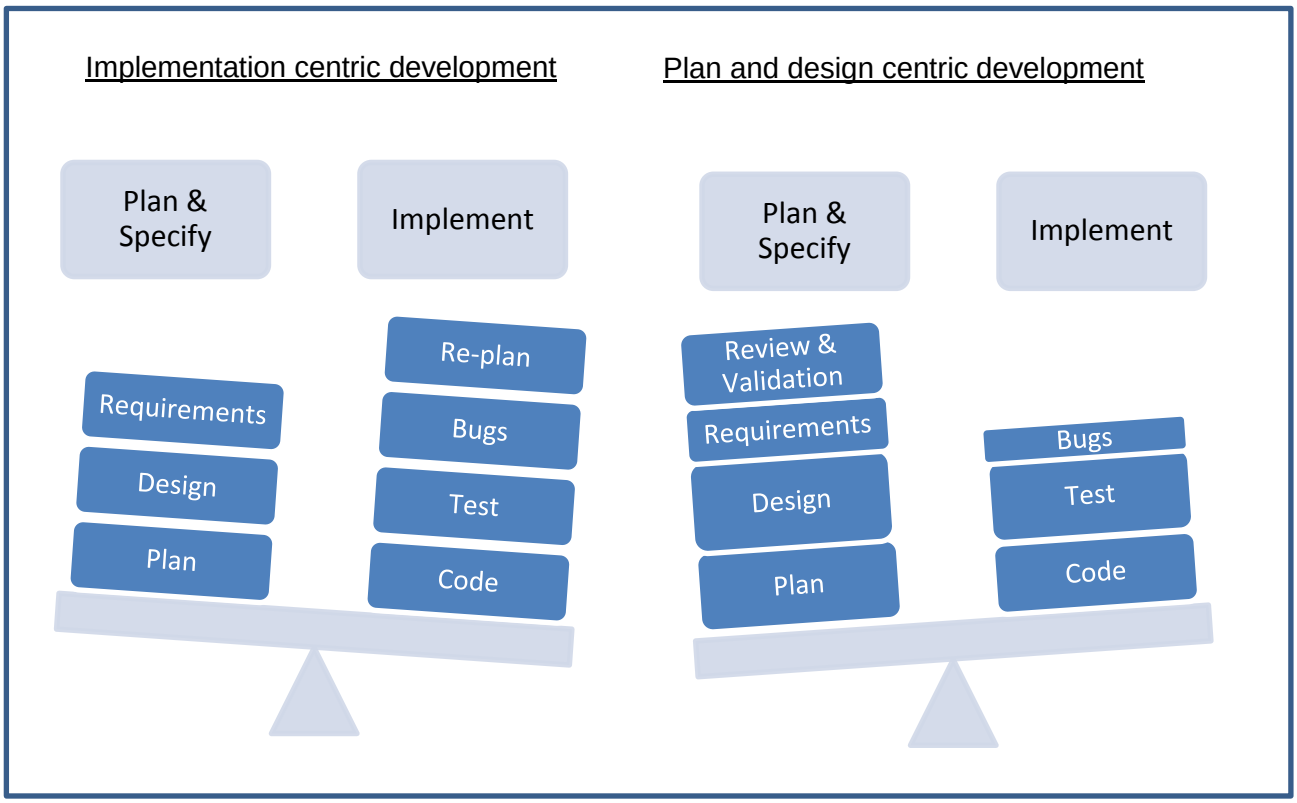

Figure 5: Focus on plan and design

4 Metrics

Metrics are key performance indicators used to gauge and follow/up system development status. It is important to have early feedback from customer to the supplier, and to be able to assess progress for the OEM.

Metrics can be characterized regarding their scope and the aspect covered.

4.1 Scope and Representation of Metrics

A metric is calculated on the basis of an entire system specification or parts thereof. Examples of delimitations include

- Architecture part

  The metric concerns a particular architecture or parts thereof
- Requirement set

  The metric concerns a particular requirement set
- Project

  The metric concerns a particular project

A hierarchy of metrics based e.g. on the structural hierarchy of the system (system, subsystem, etc.) is conceivable. Depending on character, metrics can in a tool or in a report be visually represented as lists, flags and colors in addition to plain numbers.

4.2 Aspects Covered

Aspects covered by metrics can be categorized according to purpose, e.g. quality or safety. Examples (without categorization) include

4.2.1 Qualities and Correctness

- Inconsistency
- Dependability
- Relation to requirements attributes (SMART, ISO 26262 attributes)
- Assessment of requirements formulations
- Cost

4.2.2 Progress and Completeness

- ”Technical debt” as a function of future maintenance, missing testing, missing documentation, etc., for a given system of certain complexity
- Functionality (sometimes denoted “Function Points”)
- Number of completed analyses, completed artifacts or number of reviews
- Number of change requests calculated as a retrospective analysis
- Requirements validation, verification progress

4.2.3 Risk and Impact

- Impact of monitored system on production volume, income or cost
- Number of identified project risks
- Number of handled product risks
- Appropriateness of method as a function of impact, potential value and return on effort gives the ”weight” of the metric

Most of these metrics can be tracked over time to give more benefit, in terms of revealing trends.

When considering what metrics to employ in given situation, metrics that may cause sub-optimal development should be avoided.

4.3 Categorizing Metrics

- W: Within scope
  Project Synligare is not addressing change management
- D: Demonstrable
  Deemed feasible within the time and resources of Project Synligare.
- I: Eases work with ISO 26262
  Supporting safety work, the requirements engineering efforts mentioned in ISO 26262 or general understanding of the system.
- R: Reduced development and verification time
  Supports development and verification in a novel way which is not synthesis
- N: Eases consistency check
  Supports the effort of checking consistency between implementation and specification

Scenarios

- 1: Requirements and Features
- 2: Requirements and Functions
- 3: Requirements and Hardware Components
- 4: Functional Allocation
- 5: Feature Tree with Increments and Products
- 6: Filtering
- 7: High-Level Filtering from Project Level
- 8: AUTOSAR Generation and Diff
- 9: RFQ
- 10: Progress
- 11: Integration
- 12: Annotation for SIL, Optional and Product Variants
- 13: High-Level Quality Measurements and Views
- 14: Project Earned Value over Schedule

Category	Metric	Type	Scenario	Criteria
Qualities and Correctness	Inconsistency	Aggregate including subjective values	1;2;3;4;6;7;8;11;12;13	WN
	Dependability	Aggregate including subjective values	1;2;3;4;6;7;9;12;13;14	WDI
	Relation to requirements attributes	Function of model attributes	1;2;3;4;6;7;8;9;10;11;12;13;14	WDIR
	Assessment of requirements formulations	Complex function of model attributes	1;2;3;4;6;10;12	WIR
	Cost	Aggregate including subjective values	1;2;3;5;6;7;9;12;14	W
Progress and Completeness	Technical debt	Aggregate including subjective values	1;2;3;4;5;6;7;8;9;10;11;12;13;14	WDIR
	Function points	Aggregate including subjective values	2;4;5;6;7;10;12;13;14	WD
	Number of completed work products	Function of model attributes	1;2;3;4;5;6;7;8;9;10;11;12;13;14	WDIR
	Number of change requests	Function of model attributes	1;2;3;4;5;6;7;8;9;10;11;12;13;14	WDI
	Number of project risks and product risks handled	Function of model attributes	1;2;3;4;5;6;7;9;10;11;12;13	WDI
	Progress of requirements validation or verification	Function of model attributes	1;2;3;4;5;6;7;8;9;10;11;12;13;14	WDIR
Risk and Impact	Impact of monitored system	Aggregate including subjective values	6;7;13;14	WDR
	Appropriateness of method	Aggregate including subjective values	6;7	WR

5 Views

Views represent the system under development from a certain perspective or viewpoint, see [5] (former IEEE1471). By collecting relevant elements in a view, adequate information for a certain role in a certain method or process phase can be presented. Figure 6 shows examples of viewpoints onto a system model, which would result in several useful views.

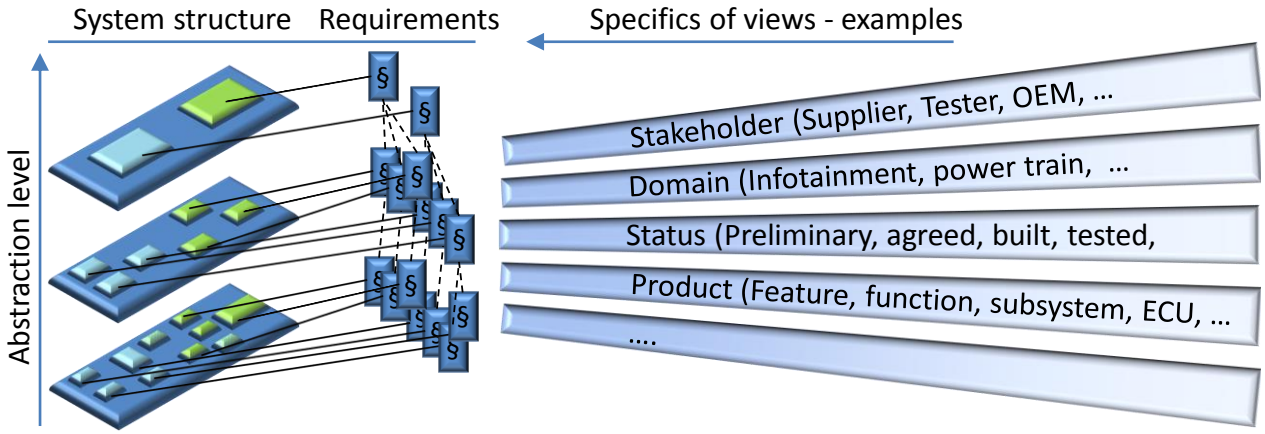

Figure 6. Examples of viewpoints on a system model

Below, some of the identified views will be presented. A view can be seen as a viewpoint, i.e. a specific subset of the system specification, combined with a layout and optional filters.

5.1 Layout

A view can have two types of layout, namely view-specific and grouped.

A view-specific layout is deliberately made for a viewpoint to make the structure and elements recognizable. This is appropriate for navigating the specification and quickly finding the appropriate information. For example, a particular type of graph, such as a fault tree has a specific layout which contains information. In such cases, the view has this view-specific layout.

Any given view may be restructured to display elements grouped according to certain categories. For elements represented in a list, grouping can be thought of as sorting. For elements represented in a graph, grouping concerns deliberate placing of elements such that elements of the same category are close to each other. There are multiple ways of choosing categories, such that there are several ways of grouping a given specification.

- Responsible
  Group elements according to which user is responsible for them
- Attribute value
  Group elements that have the same value for a given attribute
- Relations
  Structure elements so that elements that are related are close to each other
- Allocation
  Group elements according to what “function”/”feature”/”system architecture element” they are allocated to
- Abstraction level
  Group elements according to their abstraction level

5.2 Filtering

Any given view may hide elements that are not included in filtering criteria. Examples of categories that may be used for filtering include:

- Increment
  Show only elements included in a specific development increment
- Responsible
  Show only elements under the responsibility of a specific user, department, etc.
- Category
  Show only elements of a certain category, e.g. only ECUs
- Baseline, variant, versions
  Show only elements that are relevant for a certain baseline, variant, versions. This category is often more than a filter, since complex variability resolution rules are usually needed.
- Dependability
  Show only elements related to a specific ISO26262 Item
- Sorting on attribute value
  Show a given number of elements with certain values for a given attribute
- Allocation
  Show only elements that have a relationship with any of a given set of elements
- Type
  Show only elements of a certain type or category
- Search
  Show only elements that match a search criteria
- Difference
  Show especially the elements that constitute the difference between versions of a specification

Dedicated viewpoints for some of these categories are partly overlapping with the filtering concept. For example, a dependability viewpoint would collect ISO26262 artifacts in a dedicated view suitable for safety engineering. The dependability filters only manipulate existing views defined for other purposes.

5.3 Requirement Viewpoints

The Local Requirement View shows an individual requirement and its attributes. An example of an attribute is status which can have the values New, Reviewed, Implemented, Verified, Validated, Baseline or Clustered for release. Together with appropriate filters, the local requirement view can be used to see the elements involved in verifying the requirement, especially for requirements that are verified by verification of derived requirements.

The Regional Requirement View shows a group of requirements. This group can be selected in multiple ways

- According to features, functions or components
- According to order and sorting with regard to multiple aspects

There are specific uses of the Regional Requirement View that shows the derivation trace of a requirement or the children of a requirement across the abstraction levels.

The Global Requirement View shows all the requirements. While this view may include too many elements to give detailed information, it may reveal overview and structure in the requirement specification through the use of appropriate layouts and filters.

The Use Case View shows given use cases.

The State Machine View shows state machines.

The Simulation Model View shows simulation models.

5.4 Architecture Viewpoints

The System Architecture View shows system architecture and requirements as well as the allocation of requirements to system architecture elements. This view is used to define black boxed functions and to handle their integration. The view is also used for the definition of confidential and shared subsets of the specification.

The Feature View shows features and related functions.

The Functional View shows functions and requirements as well as the allocation of requirements to functions and related elements.

The Component Allocation View shows elements and their relations to system architectural elements.

The Software Architecture View shows the software architecture elements and related elements including requirements.

The Physical View represents the physical appearance of the system and requirements allocated to the different system elements, as well as optionally various modes, physical forces, static measurements and descriptions on dynamic behavior.

The EE Architecture View gives an overview of the EE architecture, gathering metrics regarding how it scales and performs. It can also show the availability and reliability of required components.

The Variability View shows features and alternative configurations as well as relations between different configurations. This view can be used to show feature increments, product family commonality, baselines, variants and versions, as well as to manage reuse.

The Model Validation View shows the most abstract requirements, such as the wishes of stakeholders together with related elements to aid model validation.

5.5 Safety Viewpoints

The Safety Goals View shows all safety goals for a certain item.

The Safety-Related Components View shows all components for a certain item.

The Functional Safety Concept View shows requirements and elements that make up the functional safety concept.

There are various safety analysis views that enable and show Fault Tree Analysis, FMEA, etc…

5.6 Quality Viewpoints

The Quality Goal View shows aggregated quality metrics with respect to quality goals.

The Process Metric View shows aggregated metrics with regard to the development process quality, such as releases over the plan and the number of errors, etc...

The Design Rule Compliance View shows elements and subsets of the specification that do not comply with given design rules.

5.7 Capabilities / Other Viewpoints

The Diagnosis View shows diagnosis-related elements.

The Software Download View shows elements related to software download.

The Power Management View shows elements related to power management.

The Dynamic Behavior Analysis view shows elements related to response time and other timing analysis.

5.8 Planning Viewpoints

The Planning View is necessary to give engineers and project managers understanding of the current state of work. Effective visualization gives instant overview and highlight problem areas. High level metrics are important data for such viewpoints.

- Trends
  Regardless of viewpoint, trends are needed to predict status on specific milestones
- Feature and Function Anatomy
  Status of features and functions aggregated according to containment hierarchies

Combined with filters for baselines, variants and versions, the planning view can be more focused.

5.9 Categorizing Viewpoints

Criteria are applied on the viewpoints to identify the most useful ones. In applying the criteria, it is assumed that some appropriate filters are available for the viewpoint. Especially, most viewpoints could be combined with filters to combine two versions of the specification. Further, legacy properties could be highlighted in any view if such information is available.

- W: Within scope
  Project Synligare is not addressing change management
- O: Eases OEM-Tier-1 collaboration
  Enables declaration of black boxed functions or supports integration. Enables status reports.
- D: Demonstrable
  Deemed feasible within the time and resources of Project Synligare.
- I: Eases work with ISO 26262
  Supporting safety work, the requirements engineering efforts mentioned in ISO 26262 or general understanding of the system.
- R: Reduced development and verification time
  Supports development and verification in a novel way which is not synthesis
- C: Eases component synthesis
  Enables the automatic generation of a skeleton component description from other elements
- V: Eases requirement and verification & validation structure synthesis
  Enables the automatic generation of requirement boilerplates, of skeleton verification and validation case descriptions or of relations between such elements
- N: Eases consistency check
  Supports the effort of checking consistency between implementation and specification

- F: Enables fast feedback
  Aggregates an appropriate set of elements so that overview and understanding is obtained, for example in the form of metrics

Scenarios

- 1: Requirements and Features
- 2: Requirements and Functions
- 3: Requirements and Hardware Components
- 4: Functional Allocation
- 5: Feature Tree with Increments and Products
- 6: Filtering
- 7: High-Level Filtering from Project Level
- 8: AUTOSAR Generation and Diff
- 9: RFQ
- 10: Progress
- 11: Integration
- 12: Annotation for SIL, Optional and Product Variants
- 13: High-Level Quality Measurements and Views
- 14: Project Earned Value over Schedule

Category	**Viewpoint**	**Example uses of the viewpoint with appropriate filter or layout**	**Scenario**	**Criteria**
Requirement Viewpoints	Local requirement	Verification strategy view	6	WDIR
	Regional requirement	Derivation trace view Decomposition view Impact analysis view	1;2;3;6;8;11	WDIRCVNF
	Global requirement	Overall structure Overview of specification	1;6	WDINF
	Use case		9	WDCV
	State machine			WODICVNF
	Simulation model		9	WODIRCVNF
Architecture Viewpoints	System architecture	Overview of system elements System structure view Black boxing of functions Integration view	6;9;11	WODIRCVNF
	Feature		1;6;9;12	WDIVF
	Functional	Functional safety requirements	2;4;6;8;9	WDIVF
	Component allocation		3;4;6;9;11	WODIRCVNF

	Software architecture	Consistency view for AUTOSAR Software safety requirements	6;8;11	WODIRCVNF
	Physical		6	WIRVF
	EE architecture	Reliability Availability of components	5;6;12	WODIRCVNF
	Variability	Baseline Reuse Product family Increments	5;6;9;10;11;12	WODIRCVNF
	Model validation		6;12	WODIRN
Safety Viewpoints	Safety goals		2;6;12	WODIVN
	Safety-related components	Freedom from interference	3;6;12	WODICVNF
	Functional safety concept		2;6;12	WODIV
	Safety analysis FTA		6;12	WODICVN
	Safety analysis FMEA		6;12	WODICVN
Quality Viewpoints	Quality goal		7;13	WODIRVNF
	Process metric		7;10;11;13;14	WODF
	Design rule compliance			WODNF
Capability & Other Viewpoints	Diagnosis			WOCV
	Software download		11	WOCV
	Power management			WICV
	Dynamic behavior analysis		9	WIRNF
Planning Viewpoints	Planning	Project and plan accuracy	6;10;	WODRF
	Overview of planning-related metrics	Trends Status reports	6;7;10;14	WODRF
	Feature and function anatomy		6;10;11	WODRCVNF

6 Information and Representation

The intention with Synligare is to provide means for views, metrics, exchange, analysis and synthesis. This relies on a system representation that carries the required information. Below are examples of elements and relations.

The idea with the exchange models is to unify the definition of tasks, requirement, design and other important information. It is useful in order to:

- Avoid misunderstandings between project members, teams etc.
- Simplify data exchange between different tools
- Simplify for metrics and views
- Make projects more visible

6.1 Project-Related Information

Information related to a specific project is useful to support planning. By keeping track of tasks and things like start and due dates, net duration (i.e. effective time) and related work products, it is possible to track progress and make estimates. Figure 7 shows how such information can be used to show earned value graphs using the script engine Jenkins.

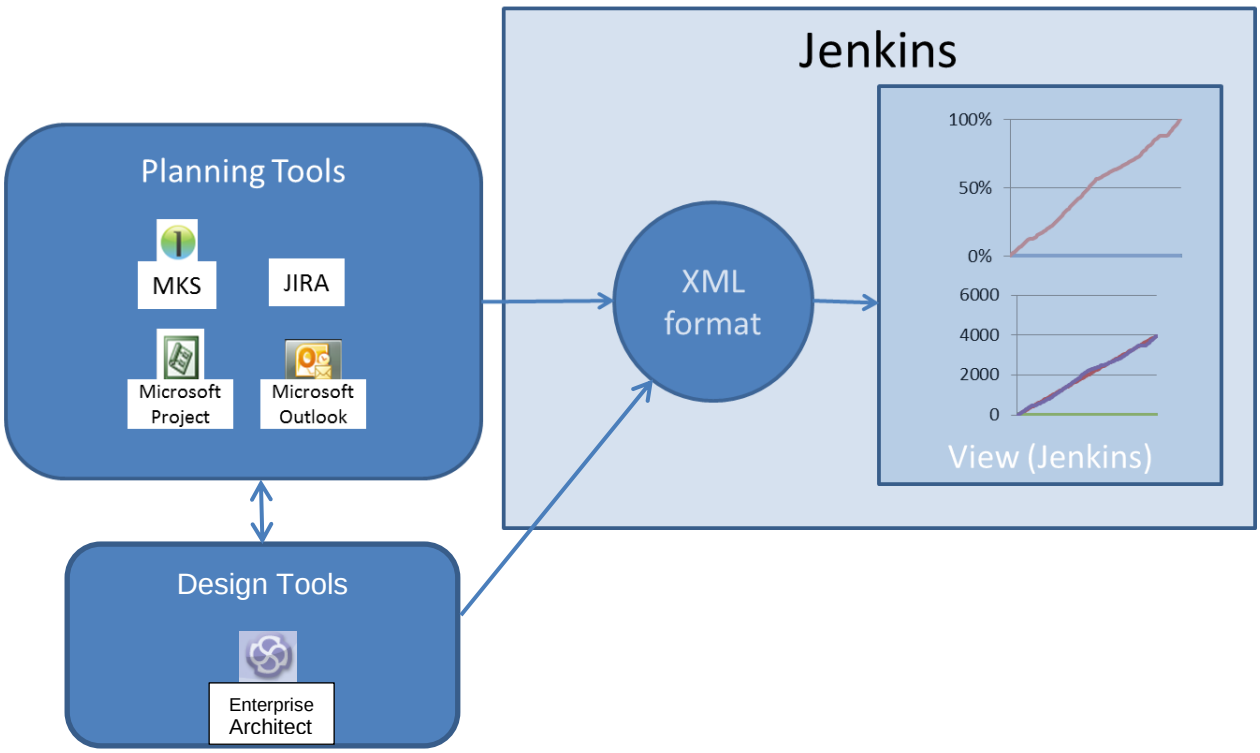

Figure 7. Use of planning and status information to follow-up progress

The task model includes attributes, dependencies and other definitions see Figure 8.

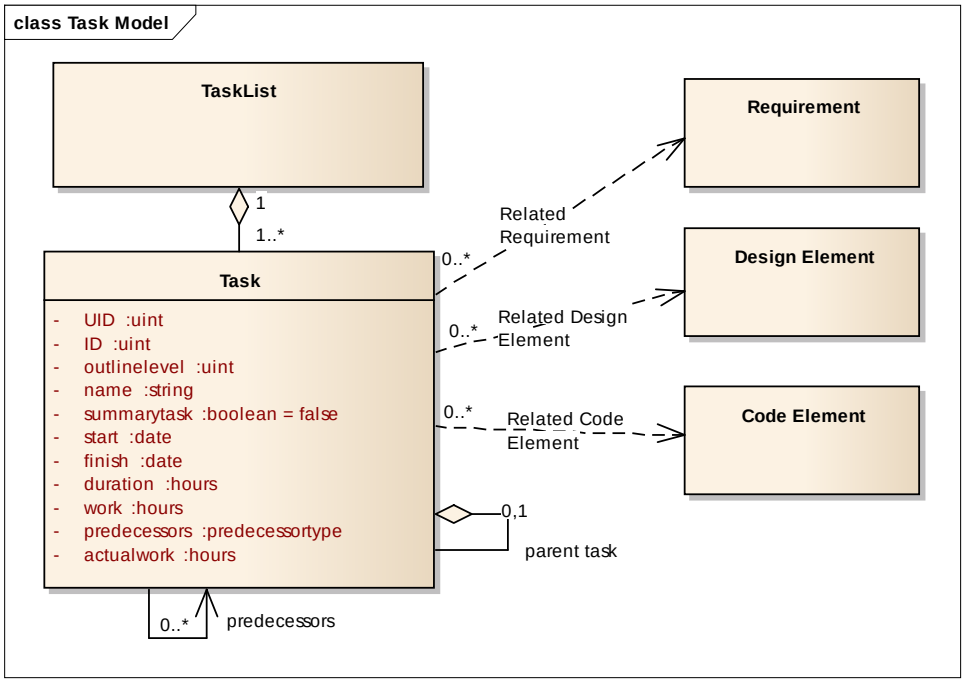

Figure 8. Relevant information for keeping track of a project task

By defining an XML schema for the task model, project status information can be exchanged between tools. Each task has a life cycle resembling to Figure 9.

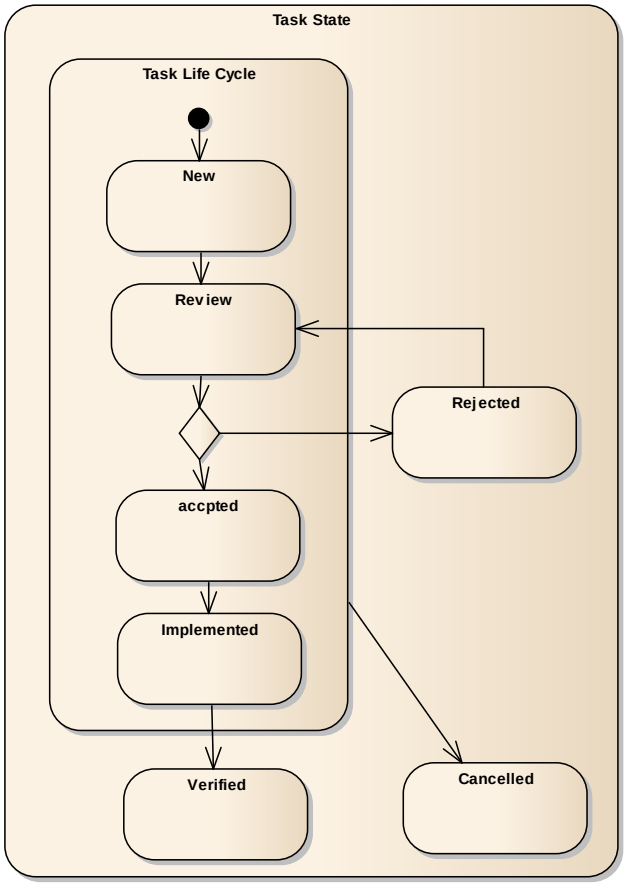

Figure 9. States useful to track the progression of a task

This means that every requirement, design element and finally code fragment also has a similar life cycle. The planning and the work therefor always have to be closely linked together. It is tempting to put everything in the same model. But there are many things to consider before this is feasible.

6.2 Product Related Information

To understand and exchange product information, requirements and product structure has to be represented adequately along with relevant annotations. AUTOSAR provides means to represent software architecture and details of the computational architecture while EAST-ADL represents features, functions and topology.

6.3 Exchange Format

There are several techniques available to collaborate among tools and exchange data. For example, requirements can be exchanged with ReqIF and product structure can be exchanged using the ISO10303 STEP standards. UML and SysML are general modelling approaches that allow models to be represented and serialized according to the XMI standard.

In order to bring semantics to exchanged information, i.e. make exchanged information well-defined and understandable, the exchange format needs to be more specific. An ontology covering requirements, features, functions, components and their relations is necessary. Compare with e.g. MS Word .docx format, which allows documents to be exchanged but does not control whether there are chapters for abstract, intro, and conclusions. For this reason AUTOSAR/EAST-ADL is useful, since it allows the relevant elements and relations to be captured in the same framework and according to an established metamodel. The corresponding exchange format is ARXML and EAXML.

7 Collaboration Aspects

Improved transparency of the development process requires OEM and supplier to share more information, which may reveal sensitive aspects. Working with models means that the information is more accessible and useful from a technical point of view which further emphasizes the importance of settling liability, confidentiality and IP issues.

7.1 Liability and Change

The overall goal of using models and defining clear views and metrics is to reduce development time and risk, improve safety, quality, performance and functionality. While reaching these benefits, it is important to understand the drawbacks or new aspects that come into play.

When models are used, supplier and OEM typically expose more of their system definition and a more precise view of the system definition. This means that

- Changes are potentially more expensive for the customer (typically the OEM or tier i+1)
  Precise definitions shows the extent of changes more explicitly
- Mistakes in the specification become explicit
  From a liability point of view, it is clear who has the responsibility

7.2 Collaboration on Different Levels

Efficient collaboration is necessary to succeed in a project. For an OEM like Volvo or supplier like Autoliv the organization in a big project is extensive, including a lot of teams in several plants. This means that the collaboration aspect within the OEM or within the supplier also needs to be more accessible and useful. Collaboration between the Tier i and Tier i+1 generally face the same problem, although scale goes down along the value chain.

7.3 Requirements vs. Solutions

A common issue in collaboration within and across organizations is the difference between requirements and solutions. This is problematic, because

- It is not possible to know from model content if a requirement or solution is defined
  When requirements are no longer textual and called requirement, it is not clear what aspects of a model are normative vs. informative.
- Depending on roles, the same model may be a requirement, solution or offer. Some examples follow below.
  Offer: The OEM may receive a model as a suggestion from a supplier.
  Requirement: After accepting the suggestion, it cannot be changed and is considered as a requirement from the OEM.
  Solution: Similarly, a model may represent a solution (one out of many possible) to a control problem.
  Requirement: Once the control engineers have verified the model, it becomes a requirement for the software engineers.
- Depending on engineering phase, the same information may represent a requirement, solution or suggestion
  Solution: In the early engineering phase, a model may represent a solution (one out of many possible) which is being integrated in an overall system description and verified.
  Requirement: The same model may be deemed as a requirement to be respected during further system refinement.

Without explicit means to express the interpretation of model content, exchange of information will become unclear.

In practice, an entire model is not likely to be normative. It should thus be possible to annotate individual model elements as normative or informative. This way, the critical aspects e.g. for integration are respected. Parts of the model that are included only to provide context, or parts that could be solved in alternative ways are left open to the receiver of the model.

As was described above, models describe and formalize engineering information. Traditionally, requirements were used for this purpose, and their role can thus shift. Any model or part of a model marked as requirement marks which properties need to be verified explicitly.

8 Tooling Aspects

Tooling prototypes will be developed to demonstrate and validate model exchange, visualization, traceability and follow-up. The context will be development collaboration between OEM and Tier-1, but can be applied also on collaboration between Tier-i and Tier-i+1 as well as inside an organization.

8.1 Identification of Views and How They Can be Populated

Views may be identified by the engineer and populated by executing filters and scripts on the tool's models. Views may also be fixed and hardcoded into the tool.

The former provides more flexibility while the latter provides more advanced and purpose-specific views.

8.2 Tooling Requirements

The tool effort in Synligare is subject to a set of requirements, some of which has been identified in this document and in the project description.

- Technology Readiness Level 6
  Tooling shall reach TRL6. Applying the NASA definition of technology maturity on tools would suggest that TRL6 requires technology to be proven on a realistic setting.
- Usability
  The tooling shall have an acceptable level of usability. Acceptable level means in Synligare that tooling shall be sufficiently user friendly to allow application of the tooling on a realistic example.
- Tool qualification
  Tool qualification needs shall be considered, and an initial assessment of each tool shall be performed.
- Feasibility
  The tooling shall be feasible to use in a realistic OEM-tier-1 scenario. To assess feasibility, criteria like scalability, completeness, user acceptance would be considered.
- Support for comparing differences
  Analysis and presentation of difference between two model versions shall be supported. This is needed to identify model differences between engineering steps and between hand-overs from one stakeholder to another.
- Navigation
  When exploring models in a tool, it shall be possible to navigate up and down model hierarchy and along direct and indirect references

These requirements are initial and stated independently of the detailed tooling concepts. This means that they are subject to change and refinement before being applied on the concrete tooling.

8.3 Tool Platforms

Synligare tooling will be based on two platforms, SystemWeaver from Systemite and ARTOP/EATOP that are Eclipse-based and open source. For ARTOP, the open source is restricted to AUTOSAR members, but the principle is still that the tooling is open and community

driven within the automotive context. Legacy tools will also be used and so far Doors and Enterprise Architect have been identified.

- SystemWeaver implementation
- EATOP implementation

  EATOP is an Eclipse project providing a tool platform allowing EAST-ADL models to be managed. Tool plugins co-exist on the platform and can be provided by different vendors.

- ARTOP implementation

  ARTOP is an Eclipse-based tool platform allowing AUTOSAR models to be managed. Tool plugins co-exist on the platform and can be provided by different vendors. The ARTOP development collaboration is open to AUTOSAR members.

- Legacy Tool implementation
  - Doors

    Doors is a requirements engineering tool, widely used in the automotive industry. Requirement authoring in Doors to be integrated with the Synligare approach.

  - Enterprise Architect

    Enterprise Architect is a UML tool. It will be investigated how the Synligare approach can integrate modeling with Enterprise Architect. For example, the EAST-ADL UML profile would be used and Enterprise Architect visualization support investigated. Scope could be limited to requirements, features, functions, topology and allocation. To integrate properly, export and import from Enterprise Architect to EAXML and ARXML would be necessary.

- Jenkins as manager/trigger of analyses

  Jenkins is a scripting or macro engine that allows automation of data collection and presentation. Depending on the models and tooling used, it may be used to assist the provision of views and metrics.

9 Scenarios

This section contains a set of scenarios that represent project needs. Based on the scenarios it is possible to identify concepts and guidelines as well as tool views (including dialogues and other user interfaces), metrics and services that are needed to support each scenario.

9.1 Scenario “Requirements and Features”

1. Define feature tree capturing the intended functionality of the vehicle
2. Attach requirements to Features
3. Visualize requirements per feature

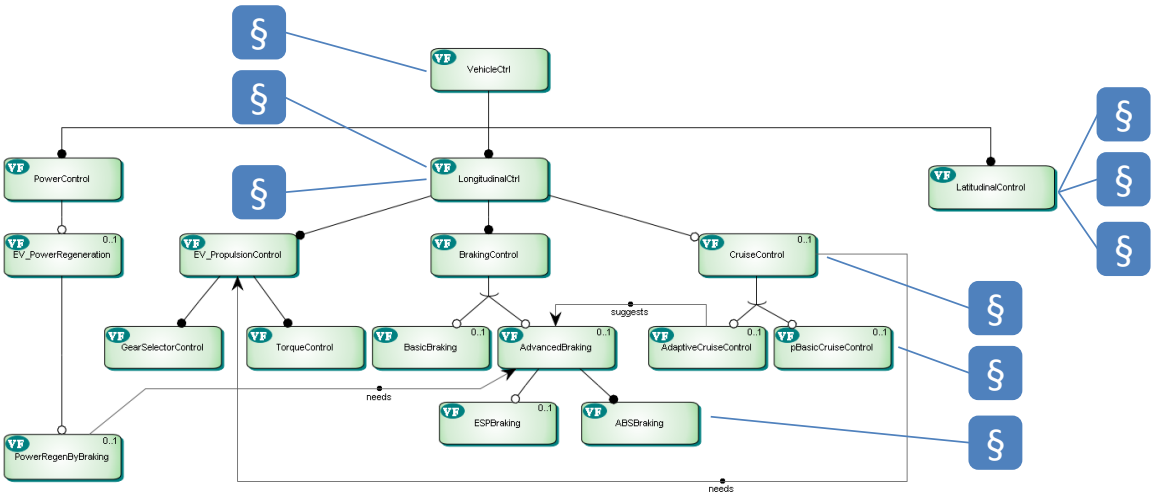

Figure 10. Example of EAST-ADL Feature Tree

9.2 Scenario “Requirements and Functions”

1. Define functional architecture capturing the intended functional solution of the vehicle
2. Attach requirements to Functions
3. Visualize requirements per function

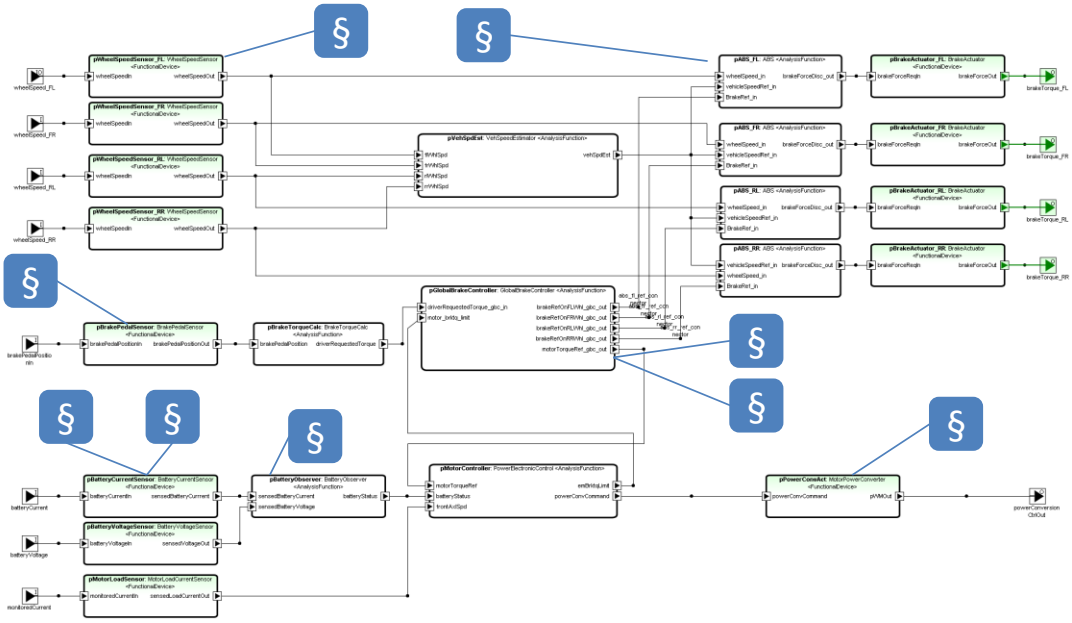

Figure 11. Example of EAST-ADL Functional Architecture

9.3 Scenario “Requirements and Hardware Components”

1. Define hardware architecture capturing the intended topology of the EE architecture of the vehicle
2. Attach requirements to components.
3. Visualize requirements per feature

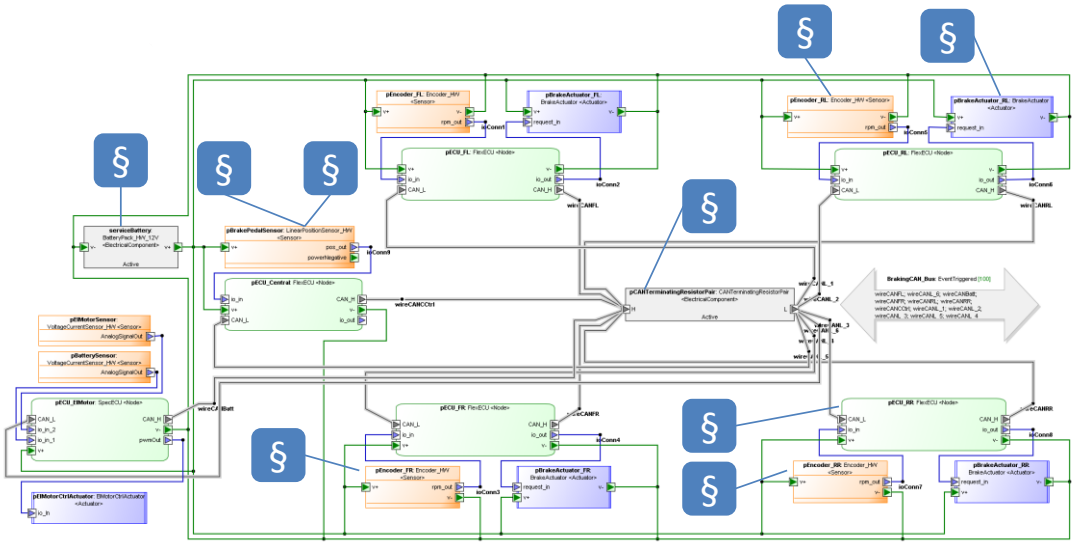

Figure 12. Example of EAST-ADL Hardware Architecture

9.4 Scenario “Functional Allocation”

1. Define functional architecture capturing the intended functional design of the EE architecture.
2. Define hardware architecture capturing the intended topology of the EE architecture.
3. Define allocation relation from function to hardware

4. Visualize allocation per function and per target

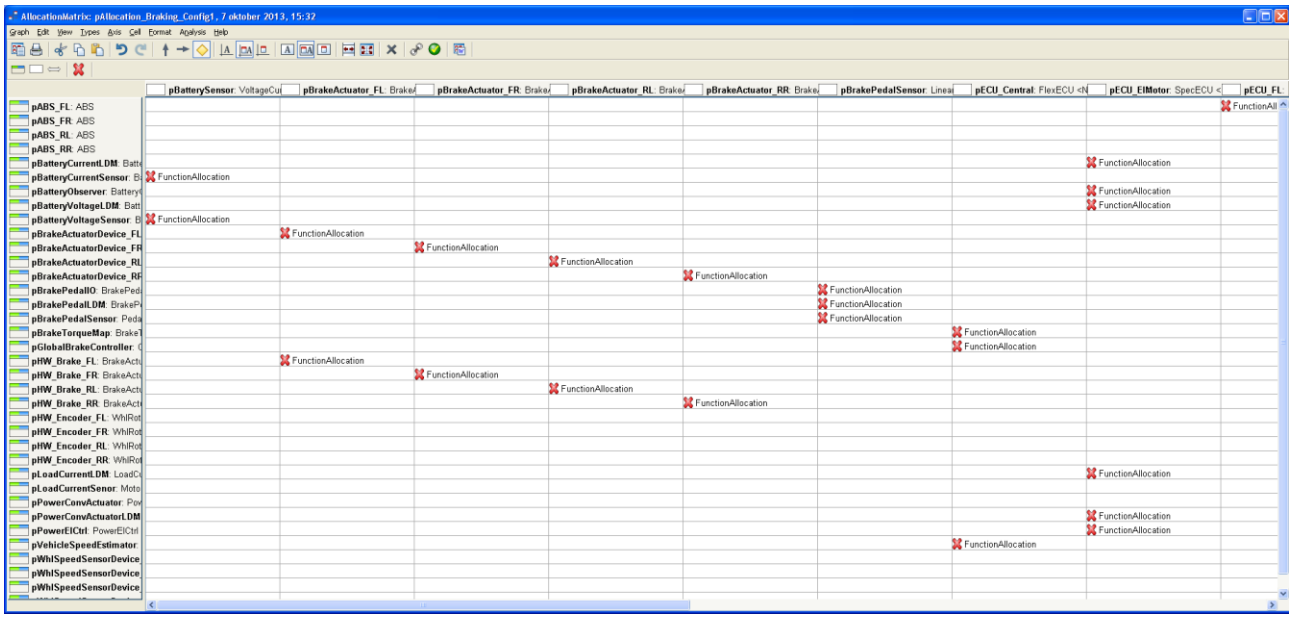

Figure 13. Example of EAST-ADL Function-to-Node allocation

9.5 Scenario Feature Tree with increment and products

1. Define feature tree capturing the intended functional of the vehicle
2. Attach features to ASIL and product, configuration and variability
3. Visualize feature trees together with ASIL, product, configuration and variability

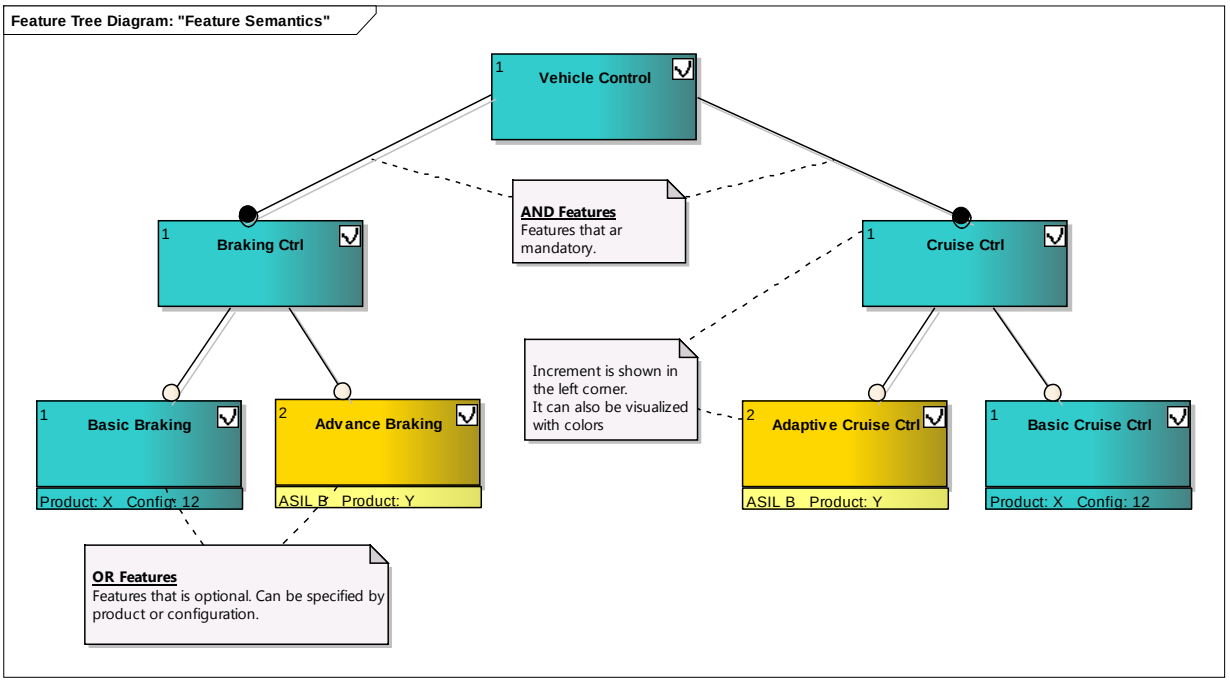

Figure 14. Example of Feature Tree with increment and products

9.6 Scenario Filtering

1. Define diagrams (below example with feature tree)
2. Attach filter(s) such as increment, product, status, cost etc.
3. Visualize filtered diagrams

When filtering is on all diagrams shall be filtered.:

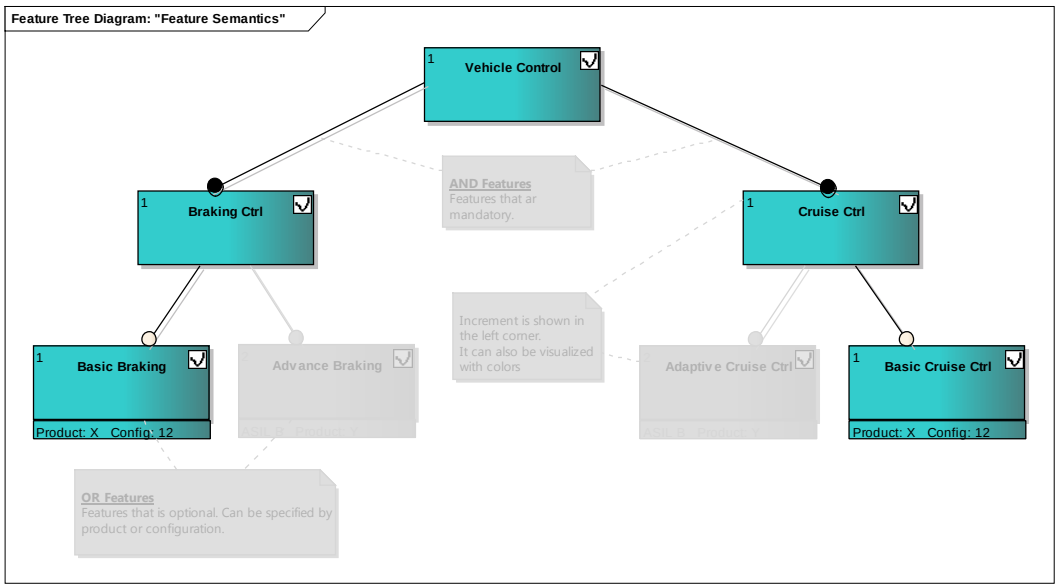

Figure 15. Example of Feature Tree filtered according to increment (only showing increment 1)

9.7 Scenario High level filtering from project level

The filtering should also be possible to make from pie charts that present essential metrics, like the figures below which shows

- Features per increment (left chart)
- Not yet accepted elements (right chart)

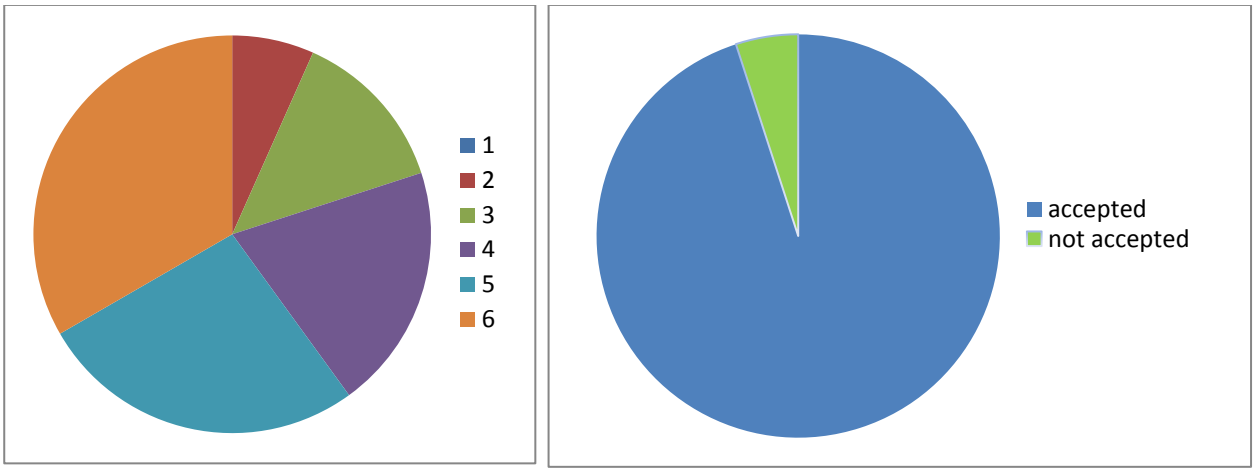

Figure 16. Example of Visualization of increment distribution and accepted elements

1. From high level metrics click on selected item

2. A more detailed table or pie chart is shown
3. Visualize filtered diagrams
4. Show detailed value(s)

This is inspired by the Virtual Project Leader (VPL) from LEQM.

9.8 Scenario Analysis

1. Define functional architecture
2. (Define topology and allocation)
3. Annotate model with analysis-specific information such as
    a. Error propagation
    b. Failure probability
    c. Cost
    d. Timing
    e. …
4. Analyse model and present results according to character of results

9.9 Scenario AUTOSAR Generation and Diff

1. Define functional architecture
2. Generate Autosar SWC hierarchy
3. Modify Autosar SWC hierarchy
4. Modify functional architecture
5. Generate Autosar SWC hierarchy
6. Recover modifications from step 3

9.10 Scenario RFQ

1. OEM defines a feature tree capturing the intended functionality of the vehicle
2. OEM defines requirements and use cases and attach these to Features
3. OEM identifies part of the functionality to buy from supplier
4. OEM defines System Model (Features, Analysis Functions, Design Functions, Hardware Components, Software Components and System Template) with certain parts only with black box view.
5. OEM sends System Model to supplier for RFI
6. Supplier responds with refined System Model
7. OEM sends System Model to supplier for RFQ
8. Supplier responds with refined System Model

9.11 Scenario Progress

1. Supplier provides partial System Model based on current status to OEM
2. (Another supplier provides partial System Model based on current status to OEM)
3. OEM assess progress by applying relevant viewpoints
4. OEM assess progress by applying relevant metrics

9.12 Scenario Integration

1. Supplier provides partial System model based on current status to OEM
2. (Another supplier provides partial System Model based on current status to OEM)
3. OEM adds partial System Model based on current status
4. OEM integrates System Model

5. OEM assess progress by applying relevant viewpoints
6. OEM assess progress by applying relevant metrics

9.13 Scenario Annotation for SIL, Optional and Product Variants

1. Define important attribute (SIL, optional and product variants)
2. Control outlook, for example: optional is shown as dashed box
3. Visualize annotated diagrams

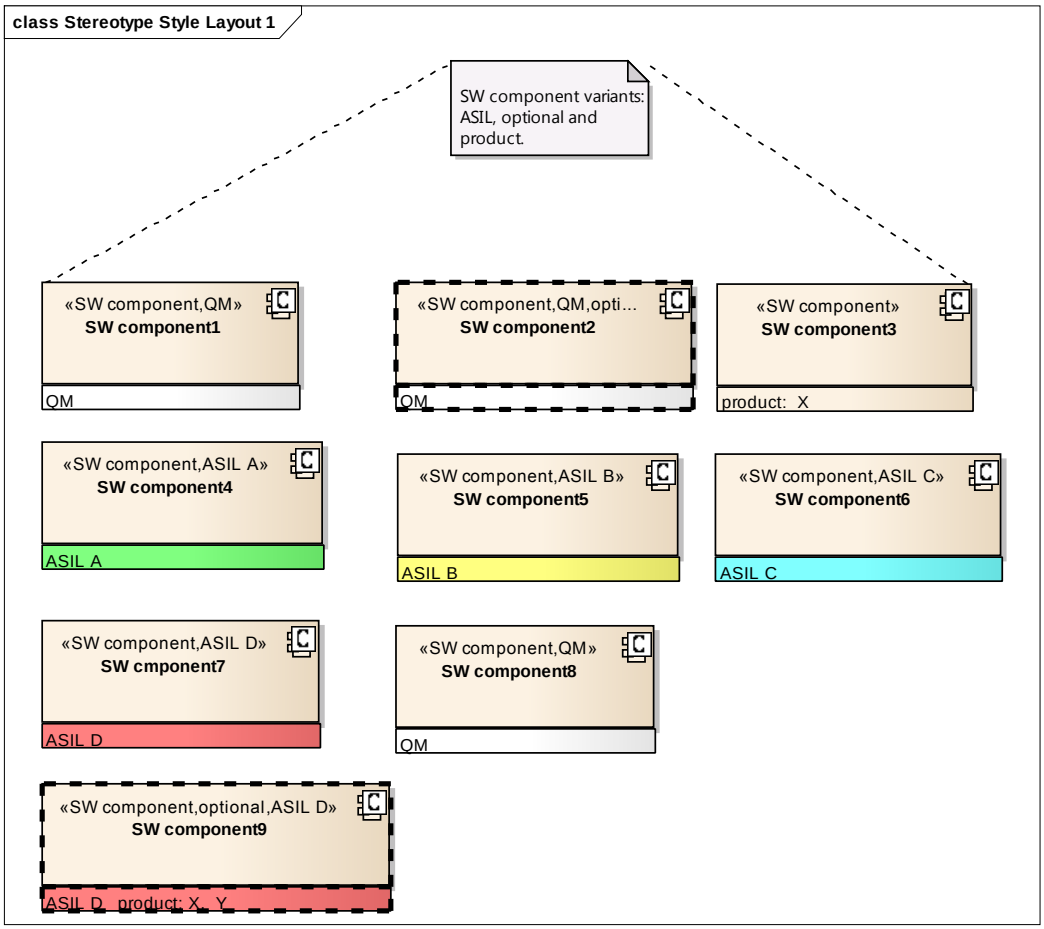

Figure 17. Example of annotated diagrams

9.14 Scenario High Level Quality Measurements and Views

Speculative and subjective metrics is combined with objective information from requirement, design, code, plans etc.

- Quality and maturity status is combined
- Quality Goals

1. Define important quality attributes / quality goals etc.
2. Define calculation functions
3. Visualize data and trends

9.15 Scenario Project Earned Value / Schedule

1. Define important project attributes like cost, earned value/schedule etc.
2. Define calculation functions
3. Define what to import from tools
4. Define export / import format
5. Export / Import data
6. Visualize data and trends

The output is shown in trend graphs. By using coloring and other means problems and risks shall be highlighted.

In the diagrams below the work progress and the artifact progress is shown.

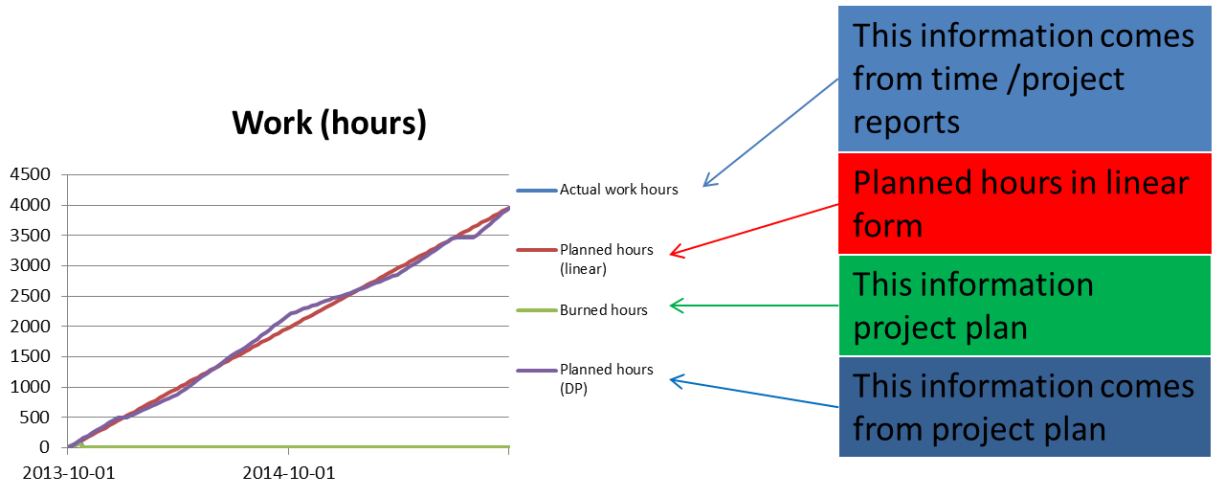

Figure 18. Example work progress diagram

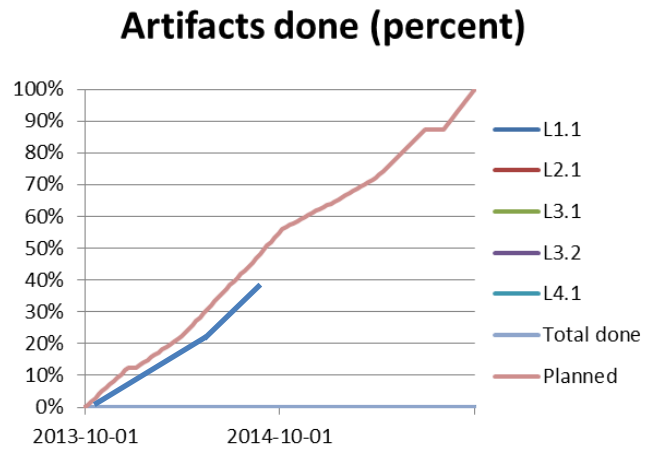

Figure 19. Example artifact progress diagram

9.16 Scenario Exchange – One-Way

1. OEM creates a system description
2. OEM exports model to EAXML

3. Supplier Imports EAXML model
4. Supplier adjusts one or several components
5. Supplier exports model to EAXML

9.17 Scenario Exchange - Round-Trip

1. OEM creates a system description
2. OEM exports model to EAXML
3. Supplier Imports EAXML model
4. Supplier adjusts one or several components
5. Supplier exports model to EAXML
6. OEM Imports EAXML model
   Updated elements replaces original elements, while maintaining links

10 Conclusions

According to the investigation in this report, Synligare concept, tool and demonstrator development will cover a selected set of views, metrics and scenarios. In doing this, the general findings related to model representations, IP aspects, exchange format, etc. will be respected. Further, focus is on supporting ISO26262 and collaboration between OEM and Tier-1. Below, we will summarize the views, metrics and scenarios to address in the project.

10.1 Views

The following views will be supported, due to their importance for OEM-tier-1 collaboration and because they are relevant for ISO26262.

- Requirements
  - Clustered on Features/functions/components
  - Showing relevant attributes
  - Multiple aspects according to selected order and sorting
- Safety
  - All safety goals for a certain item
  - Functional Safety Concept
  - Technical Safety Concept
- System and Software Architecture
  - All elements realizing a certain Feature
  - Function allocation to software architecture
- Progress
  - Requirements coverage
  - Element coverage (feature, function, …)

10.2 Metrics

The following metrics will be supported, due to their importance for OEM-tier-1 collaboration and because they are relevant for ISO26262.

- Dependability
- Number of completed analyses, completed artifacts, number of reviews
- Requirements validation, verification progress
- Impact of monitored system on production volume, income and cost

10.3 Scenarios

The following scenarios will be supported, due to their importance for OEM-tier-1 collaboration and because they are relevant for ISO26262.

- Scenario “Requirements and Features”
- Scenario “Requirements and Functions”
- Scenario “Requirements and Hardware Components”
- Scenario “Functional Allocation”
- Scenario AUTOSAR generation and Diff
- Scenario RFQ
- Scenario Progress
- Scenario Integration
- Scenario Analysis

11 References

[1] ATESST2 Consortium (2010): ATESST2 Project web site. http://www.attest.org/

[2] AUTOSAR Development Partnership (2012): AUTOSAR web site. http://www.autosar.org/

[3] EAST-ADL Association Members (2012): EAST-ADL Association web site. http://www.east-adl.info/

[4] International Organization for Standardization (2011): Road Vehicles – Functional Safety – Part 1 to 9. International Standard ISO/FDIS 26262. November 2011.

[5] International Organization for Standardization (2011): Systems and software engineering — Architecture description. International Standard ISO/IEC/IEEE 42010:2011

[6] MAENAD Consortium (2012): MAENAD Project home page. http://www.maenad.eu/

[7] Poppendieck, Mary and Poppendieck, Tom: Lean software development

[8] TIMMO Consortium (2012): TIMMO 2 USE Project web site. http://www.timmo-2-use.org/

[9] VeriSpec consortium (2013): VeriSpec - Strukturerad specifikation och automatisk verifiering för funktionssäkra fordon. FFI project 2013-01299.

A Appendix – Questionnaire 1

An interview study based on two questionnaires was conducted in order to collect needs. Below is the outcome of the first questionnaire.

Outcomes of questionnaire

This is a summary of results extracted from four responses to Questionnaire 1.

Themes

These overall themes were seen in the responses

- Communication is important to improve today's requirements engineering
  - Communication helps us to understand each other and form a common terminology. When we have good communication we coordinate efforts and ease change management. One important group to communicate with is stakeholders.
- Understanding the requirement specification makes other activities easier
  - Understanding can benefit from a good overview over the requirement specification combined with ability to focus on a subset of it. The requirement specification should have good readability.
- The problems in today's practice of requirements engineering have causes
  - Requirements engineering is neglected, which leads to stress and a tendency to carelessly following a workflow, such that the requirements data ends up being of low quality
  - Stakeholders have low awareness of the impact of requirements engineering, leading to lack of tool support
  - The inputs to requirements engineering are of varying quality, leading to an unbalanced focus and time consuming manual work
- Practitioners tend to think that the solution to problems in requirements engineering are found in a formal workflow, with a process and systematic activities, combined with competence in the engineers involved. Engineers tend to optimize their own task.

Activities

These are some of the activities that were mentioned as activities in which requirements engineering is problematic

- Reviewing
- Status
- Organizing
- Early stages of requirements engineering
- Eliciting
- Writing
- Make understandable
- Communicating
- Validation and verification according to requirement strategy

Problem

These are examples of what makes requirements engineering problematic

- Quantity of requirements
- Maintaining consistency and quality
- Reuse of requirements
- Lack of structure or understanding the reason for a structure
- Requirements can be too abstract
- Requirements can be too detailed

Views

These are views mentioned in the responses. Some of them are wishes and some are references to views that the respondent knows of.

- There should be a view for supporting integration
- There should be a view that supports back-annotation of changes into the requirement specification
- Derivation trace as support to impact analysis and change management, giving ability to relate to initial requirements
- ”Filter” & sort
  - Relevance, priority, role, domain, status
- There should be views to support working on requirements
  - Support for extracting information from requirements
  - Support for making requirements understandable
  - Support for improving the structure of the requirements specification
  - Support for improving the quality attributes of requirements
- There should be views that support a function developer in analysis of timing, scheduling and response time. Views could visualize dynamic behaviour
- Views should be flexible and user-configurable
- Views should bridge complexity in terms of quantity of requirements

Metrics

These are metrics mentioned in the responses. Some of the metrics are wishes. Other metrics are mentioned as metrics that the respondent knows of.

- There should be metrics to measure progress and quality in the early phases of development, such as the concept phase and the specification phase
- With enough understanding of the attributes to the requirements, it is possible to construct one’s own metrics
- There should be metrics for systematic accounting of V&V with a requirements verification strategy
- By measuring and comparing the amount of changes in a specification part, it is possible to indicate parts that may have low quality
- It would be nice to measure the complexity of a requirement
- It would be nice to measure the quality of the system description
- Many metrics should be shown as graphs of their value over time. Examples are: the number of requirements, status of the requirements, status as assigned by different people, the number of requirements per module and status of requirements in a given module
- There should be a metric to see if quality attributes are correctly set

The requirement status can be obtained from reviews, test results and test coverage.

Metrics that can be used to navigate to a particular subset of the requirement specification can measure that subset using the roles that defined the subset, the requirement attributes and the results from verification.

Structure

From the responses, it seems that requirements in their raw form come in multiple representations, like a list, a hierarchy, a multi-dimensional graph, use-cases, scenarios, simulation models and prototypes. These representations slightly overlap, but cannot completely replace each other. The representations belong to different levels of abstraction.

How to handle complexity

From the responses, these ideas for handling complexity were mentioned

- Database support
- Divide and conquer with respect to functions and features
- Provide better overview by using views and metrics
- Let the structure of requirements determine the product structure and development process
- Or, let the product structure and development process determine the structure to choose for requirements
- Or, if no structure can be found, make an initial design to structure the requirements around
- It is easier to handle complexity if the development project is staffed with engineers with requirements engineering competence

B Appendix – Questionnaire 2

An interview study based on two questionnaires was conducted in order to collect needs. Below is the outcome of the second questionnaire.

Outcome of the questionnaire

This is a summary of results extracted from four responses to Questionnaire 2 which was answered by Autoliv personnel.

Handling of requirements on Tier 1 level

Most problematic areas:

- OEM requirements are too many and on a much more detailed level than expected from a stakeholder requirement. This leads to additional (unnecessary) requirement work on other levels.
- Hard to handle three levels of requirements: Stake holder, sub system requirements and SW requirements
- Requirement concern is low on software level
- The implementation diverges from the requirements and it is hard to verify.
- Low test coverage due to too many requirements.
- Low reuse due to customer adaptations instead of product line architecture
- How many requirements are needed? We have either too many requirements or too few
- No common understanding that a change of an accepted requirement requires a lot of synchronized work. No good way of communicate requirement changes.
- We accept requirements from OEM that should not be accepted due to reasons like: too detailed, does not lead to good design etc.
- Implementation is done on forehand. The requirements are either too vague or come too late.
- Trace from requirement down to code is hard to make and maintain.
- Baselining not helping. Requirements come late and are changed constantly

Handling of design on Tier 1 level

Most problematic areas:

- There are too much focus on stake holder requirements and too low focus on subsystem design
- Design is not used to break down functionality, more to document
- Too little focus on design based verification
- Too much focus on structure and too little focus on behavior/collaboration.
- Coding is done without design
- Design is not used to make realistic plans in time/effort

Metrics on Tier 1 level

Most important metrics:

- Total sum of requirements in different states.
- Implement requirements per release, both delivered and planned. And trace it to design and planned work for a specific release.

Most problematic areas:

- We have different views about the project and requirement life cycle between different teams within projects and between different projects
- We don’t see a limit in number of requirements, but difficulties in defining filters. We would like to have predefined filters for each project and well defined attributes.

C Appendix – Literature Overview

Relevant work has been conducted in academia. Some has been described in the main body of the report, such as the studies that have resulted in EAST-ADL. This appendix will give examples of views and metrics mentioned in academic work and relevant innovations that can be observed in existing commercial software tools. Further, structured information models have been mentioned in academic literature.

C.1 Views in Academic Literature

In [Kruchten94] five views, called “4+1”, are presented, namely a logical view, a developer’s view, a physical view and a process view, as well as a set of scenarios. The four views have stakeholders, while the set of scenarios describe the dependencies between the four views. Conceptually, a development process can be described that starts in the scenarios to stepwise refine the views.

Requirement views are mentioned in [Lormans09] as a technique for handling requirements that are developed during the progress of a development project. Further views are considered for monitoring changes in the requirements specification and ensuring consistency. For this purpose, a requirement traceability model is proposed.

Anatomy is a view for planning and understanding a particular type of dependencies, like for example order of integration. Anatomies are described in [Taxen08].

Views are used in visual planning. Different aspects of the plan and the progress are visualized to enable stakeholders to see issues in the plan or in the progress.

Views are to be used for decision support [Russel12]. The system designer needs a view to be able to evaluate various design solutions with regard to how they satisfy requirements, how they achieve quality and how much they cost. In this paper, modeling blocks and relations are discussed and suggested. Traceability to verification cases is mentioned. A clear idea is that such views should be possible to generate from a complete system description model. Another view that is mentioned, in saying that it is helpful to bridge complexity, is the data flow diagram. This view should be combined with requirements and verification cases.

Simulation for evaluation of design solutions with the help of models can constitute a helpful view for a system designer, according to [Haveman13].

Some studies [Ambriola97, Overmyer01] discuss methods to automatically process natural language in requirements to generate corresponding semi-formal models such as data flow diagrams and communication diagrams.

In [Weber03] it is mentioned that they generate views from requirement information structured according to a model by combining search and filters. The configurations of these views are stored and can be used at a later time. The views for the manufacturer and the supplier are similar.

C.2 Views in Commercial Tools

Configurable views, decorated for easy overview are used in Medini Analyze [ikv.de].

Collections of visualized metrics are presented as views in VPL [leqm.se].

C.3 Metrics in Academic Literature

The Goal/Question/Metric paradigm [Basili92] is a method for identifying metrics. Starting with business goals, the paradigm provides templates for formulating goals. Then there are guidelines for formulating questions pertaining to those goals. There are product questions and process questions. As the paradigm continues to define metrics, they are found by asking what needs to be measured to answer the questions.

C.4 Metrics in Commercial Tools

In rCoach One, metrics are automatically generated from the requirement specification [Slingblade.se].

In VPL, metrics are used as a planning aid [LEQM.se].

C.5 Structured Information Models in Academic Literature

In [Adedjouma12], a meta-model for handling the elicitation and structuring of requirements and system architecture is described, so that traceability is achieved, as well as a model-based method for handling process models and product models together.

In model based development, recent work has argued for a model based system integration methodology [Montgomery13]. By considering integration early in the development process, integration becomes a part of the important decisions made about the design of the system. Considerations of verifiability, validation and acceptance are to be made early. They argue that the system description should be integrated before the actual subsystems are integrated. This is an example of how modelling can support early verification and validation.

Another paper on model based development describes requirements on exchange models for sharing with stakeholders [Do12]. The clarity provided by exchange models help engineers to discover problems early. They also argue that traceability benefits from having exchange models. They promote the idea of having a shared model already in the RFQ phase. By necessity, the shared model should be based on a meta-model that is expressive enough to be used for the whole development process. This is a good example of literature that is addressing the collaboration between OEM and Tier-1.

In the work [Haveman13] mentioned above, requirements on abstract models are given.

The automation aspect of structured information models has been mentioned in [Konrad03]. UML-diagrams that describe requirements written in linear time temporal logic can automatically be analyzed by model checkers.

To bridge complexity in requirement specification, especially in automotive embedded systems, it is argued in [Sternudd11] that a solution could be to define a model for knowledge in the domain of automotive embedded systems. There should be a limited vocabulary and a clear grammar. Care should be taken so that the knowledge model does not make requirements too formal. They should still be understandable by all stakeholders.

Already in 2003, [Weber03] suggested use of application-specific models for representing requirement information as well as integration of software tools to unify the requirement information flow. They suggest complementing textual requirements with illustrations, tables and artifacts that contain argumentation and test information. From these models they can generate various views by combining and filtering information from the requirements. Still they found that an important representation of the requirement information was a document-like view.

C.6 References for the Appendix

[Kruchten94] Architectural Blueprints – The “4+1” View Model of Software Architecture, P. Kruchten, IEEE Software, 1994, pp 42-50

[Lormans09] Managing Requirement Evolution using Reconstructed Traceability and Requirements Views, Lormans, M., Technische Universiteit Delft, 2009

[Taxen08] Images as Action Instruments in Complex Projects, Taxén L. and Lilliesköld, J., Internation Journal of Project Management, Volume 26, Issue 5, 2008, pages 527-536

[Russell12] Using MBSE to Enhance System Design Decision Making, Mike Russell, Conference on Systems Engineering Research, 2012, pages 188-193

[Haveman13] Requirements for High Level Models Supporting Design Space Exploration in Model-based Systems Engineering, Steven P. Haveman and G. Maarten Bonnema, Conference on Systems Engineering Research, 2013, pages 293-302

[Ambriola97] V. Ambriola and V. Gervasi. Processing natural language requirements. In IEEE Int. Conf. on Auto. Soft. Eng., pages 36–45, 1997.

[Overmyer01] S. P. Overmyer, B. Lavoie, and O. Rambow. Conceptual modeling through linguistic analysis using LIDA. In Proc. of the IEEE Int. Conf. on Soft. Eng. (ICSE), pages 401–410, 2001.

[Weber03] Requirements Engineering in Automotive Development: Experiences and Challenges, Weber, M. and Weisbrod, J., Proceedings of Joint International Conference on Requirements Engineering 2003, pages 331-340

[ikv.de] On-line. Accessed 2013-12-23

[leqm.se] On-line. Accessed 2013-12-23

[Basili92] Software Modeling and Measurement: The Goal/Question/Metric Paradigm. Basili, V. R., University of Maryland at Colleage Park, MD, USA, 1992

[slingblade.se] On-line. Accessed 2013-12-23

[Adedjouma12] Requirements engineering process according to automotive standards in a model-driven framework, Morayo Adedjouma, Delphi, 2012

[Montgomery13] Model-Based System Integration (MBSI – Key Attributes of MBSE from the System Integrator’s Perspecitve, Paul R. Montgomery, Conference on Systems Engineering Research, 2013, pages 313-322

[Do12] Requirements for a Metamodel to Facilitate Knowledge Sharing between Project Stakeholders, Quoc Do, et al., Conference on Systems Engineering Research, 2012, pages 285-292

[Konrad03] Defining and Using Requirements Patterns for Embedded Systems, Sascha J. Konrad, MSc thesis 2003

[Sternudd11] Unambiguous requirements in Functional Safety and ISO 26262: dream or reality?, Patrik Sternudd, MSc thesis 2011